

软件概要设计说明书

[TOC]

1 引言

1.1 编写目的

本文档用于说明伴游（TravelMate）出行旅游平台在系统设计阶段的总体设计方案，描述系统的业务目标、总体架构、模块划分、接口组织、数据结构、数据库方案和运行机制，为后续详细设计、编码实现、测试验证、部署交付和课程答辩提供统一依据。

本文档主要面向以下读者：

- 项目组成员，用于统一架构理解和模块边界；
- 教师与助教，用于审阅系统设计的完整性与合理性；
- 测试与部署人员，用于理解系统组成、接口关系和运行方式；
- 后续维护人员，用于快速理解系统总体设计。

1.2 背景

软件系统名称：伴游（TravelMate）出行旅游平台。

项目任务提出者：《软件工程基础实践》2026 夏课程组。

项目开发者：第 5 组（Miracle 小组）。

系统预期用户包括：

- 游客与注册用户；
- 平台内容创作者；
- 平台运营与审核人员；
- 平台管理员。

本系统参考主流在线旅游平台和内容社区的公开功能形态，在课程项目范围内整合交通与住宿预订、目的地服务、AI 行程规划、旅途社区和后台治理能力，覆盖“行前规划—行中服务—行后分享”的完整旅程场景。

1.3 定义

本文档涉及的主要术语定义如下：

1. OTA: Online Travel Agency, 在线旅游代理平台。
2. UGC: User Generated Content, 用户生成内容, 如游记、评论、图片等。
3. JWT: JSON Web Token, 用于前后端身份认证和会话校验的令牌。
4. RBAC: Role-Based Access Control, 基于角色的访问控制模型。
5. QPS: Queries Per Second, 每秒请求数, 用于衡量接口吞吐能力。
6. Mock: 模拟数据或模拟接口, 用于课程项目中替代真实商业接口。
7. RESTful API: 基于 HTTP 协议的资源风格接口设计方式。
8. Agent: 具备任务理解和多轮对话能力的智能代理。
9. JSON Output: 约束大模型返回结构化 JSON, 便于前端稳定解析和渲染。
10. RateLimiter: 自定义接口限流注解, 基于 Redis 计数器和 IP 控制高频请求。
11. Redis: 键值缓存中间件, 用于缓存、限流、延时控制和热点状态存储。
12. MyBatis-Plus: 基于 MyBatis 的 ORM 增强框架。

1.4 参考资料

本系统概要设计参考以下资料：

1. 《软件工程基础实践》2026 夏课程任务书；
2. 《5组-软件需求规格说明》；
3. 《软件项目开发计划》；
4. 《5组-软件详细设计说明》；
5. 项目现有前后端代码、接口定义、数据库脚本和原型设计稿；
6. Spring Boot、Vue 3、Vite、MySQL、Redis、Element Plus、ECharts 等技术的官方文档；
7. 主流在线旅游平台和内容社区的公开功能调研资料。

2 系统需求概述

2.1 需求规定

TravelMate 面向旅游出行全流程，要求支持用户完成从方案规划、资源检索、订单生成、行程管理，到消息提醒、内容分享和后台运营的一体化使用过程。

系统的主要输入包括：

1. 用户输入的出发地、目的地、日期、预算、人数、偏好标签等查询和规划条件；
2. 用户提交的注册登录信息、旅客信息、住客信息、订单信息、社区内容和互动行为；
3. 管理员维护的航班、火车、酒店、景点、价格、促销、敏感词和权限配置数据；
4. AI 模块调用时的对话上下文、工具参数和行程约束条件；

5. 系统监控模块采集的访问日志、异常日志和业务统计指标。

系统的主要输出包括：

1. 航班、车次、酒店、景点、本地玩乐等资源检索结果；
2. 机票、火车票、酒店、门票等订单处理结果及状态变化信息；
3. AI 生成的结构化行程单、客服问答结果和消息提醒；
4. 社区内容流、互动结果、用户主页数据和推荐结果；
5. 后台资源管理结果、审核结果、监控面板数据和系统日志信息。

2.2 业务目标

本系统的业务目标如下：

1. 构建一个覆盖“行前规划—行中服务—行后分享”完整旅程链路的综合出行旅游平台；
2. 覆盖机票、火车票、酒店、本地玩乐、AI 行程规划、AI 客服、旅途社区、后台管理和可观测仪表盘等核心业务；
3. 建立统一的用户体系、订单体系、资源体系、消息体系和权限体系，为多模块协同提供稳定基础；
4. 在课程项目范围内形成可演示、可部署、可验收、可复现的完整系统；
5. 保证关键业务链路在小并发场景下具备稳定性，满足课程对可用性和工程完整性的要求。

2.3 运行环境

本系统采用 B/S 架构，用户和管理员通过浏览器访问前端页面，前端通过 HTTP/JSON 调用后端接口，后端通过 MySQL、Redis 和外部 AI 服务完成数据存储、状态控制与智能处理。

系统运行方式支持正常运行、维护运行和降级运行三种状态：

1. 正常运行状态下，平台提供完整的资源检索、AI 行程生成、下单支付、社区互动和后台管理能力；
2. 维护运行状态下，后台可进行资源批量更新、数据库维护和配置调整；
3. 降级运行状态下，当大模型接口或外部服务异常时，系统切换为本地模板、本地数据或明确的重试提示，保证不依赖该服务的主流程仍可访问。

2.3.1 设备

运行本系统所需的硬件设备包括：

1. 开发与测试使用的个人计算机，用于前后端开发、联调和本地部署；
2. 可运行现代浏览器的桌面终端或移动终端，用于访问用户端和管理端页面；
3. 本地或局域网部署的 MySQL 与 Redis 运行主机，用于业务数据持久化和缓存服务；

4. 课程验收时用于展示和部署的普通 PC 环境，用于现场演示和答辩。

系统可使用智能手机或平板设备验证响应式页面，也可使用外接显示器展示管理后台和可观测仪表盘。系统不依赖专用票务服务器、GPU 集群或生产级多节点环境，强调轻量、可复现和课程场景下的可部署性。

2.3.2 支持软件

本系统依赖的软件环境包括：

1. Windows 或其他支持 Java 与 Node.js 的操作系统；
2. JDK 21；
3. Maven；
4. Node.js 20.19+ 或 22.12+ 与 npm；
5. MySQL 8.0；
6. Redis 6.x 或兼容版本；
7. Spring Boot 3.5.13；
8. Vue 3 与 Vite 8.2.2；
9. MyBatis-Plus 3.5.7；
10. Element Plus 2.6.3；
11. ECharts 6.1.0；
12. Nginx 1.20+；
13. Postman 或 Apifox；
14. JMeter；
15. Git 代码托管工具；
16. DeepSeek 或同类 OpenAI 兼容大模型接口服务；
17. Playwright 1.62.1，用于前端端到端测试；
18. Java HttpClient，用于访问火车公共余票数据源；
19. Commons CSV 1.11.0，用于管理后台资源 CSV 导入。

2.4 设计约束

本系统设计需遵循以下约束：

1. 项目属于课程实践，应优先保证核心业务真实、完整、可验证，不追求商业系统规模的无限扩展；
2. 不接入真实支付、真实航司库存或真实酒店分销系统，采用模拟支付和业务数据建模保证流程闭环；
3. 四名成员并行开发，要求接口契约、数据命名、主外键关系和状态机规则高度统一；
4. 关键流程必须考虑小并发稳定性，避免库存超卖、订单状态错乱和权限越权；
5. AI 模块必须具备可解释性，Prompt、Session、结构化输出约束和降级策略需要能够单独说明；
6. 需求、设计、实现和测试采用统一术语、接口契约与追溯编号；

7. 设计应兼顾课程扩展目标，包括智能服务、可观测性、UI/UX 和可扩展功能；
8. 前端单页面应用需支持路由懒加载与资源分包，以适应不同网络环境下的访问速度要求；
9. 地理位置类数据统一采用标准经纬度格式；
10. 订单金额在后端实体中使用 `BigDecimal` 按“元”为单位存储与传输，前端展示时统一格式化为人民币金额。

2.5 功能需求

系统按业务职责划分为五个核心子系统，共同构成完整的旅游平台。

1. 大交通票务子系统

支持机票搜索、航班详情、航班预订、火车票查询、席别选择、中转推荐、价格趋势、火车候补、公共余票同步状态、常用旅客管理、订单状态流转、模拟支付、退款申请/审核和订单凭证查看。

2. 酒店与目的地服务子系统

支持酒店搜索、房型展示、房务预订、防超卖控制、景点门票预订、本地玩乐推荐、订单凭证展示、订单完成后评价、图文评价、商家回复和举报处理。

3. AI 智能规划与客服子系统

支持用户输入预算、目的地、天数、人数和偏好后生成结构化行程；支持 AI 客服通过多轮对话回答旅行问题，并结合站内资源数据给出建议；支持消息通知、未读红点和历史行程查看；当大模型接口超时、额度异常或结构化输出失败时，系统自动降级为预置行程模板或推荐资源回退结果。

4. 旅途社区与用户中心子系统

支持图文内容发布、目的地/话题标签、瀑布流浏览、关注流与推荐流切换、点赞收藏、评论回复、关注关系、用户主页、账户安全和行为记录。

5. 管理后台与可观测子系统

支持运营人员和管理员完成资源库管理、CSV 批量导入、库存干预、优惠券维护、内容审核、用户启禁用、举报处理、退款审核、订单流水查看、角色权限控制、系统日志审计、业务统计和可观测大屏展示。

系统完整业务闭环可概括为：

用户业务按行前、行中和行后三个阶段组织。行前包括 AI 行程规划以及交通、酒店、景点和本地游查询预订；行中包括库存校验、订单支付、凭证查看、消息通知和 AI 客服；行后包括评价、游记发布、点赞、评论和关注。后台治理贯穿全流程，负责资源维护、退款审

核、内容审核、举报处理和日志统计，使各业务场景均形成可查询、可验证、可追溯的结果。

2.6 非功能需求

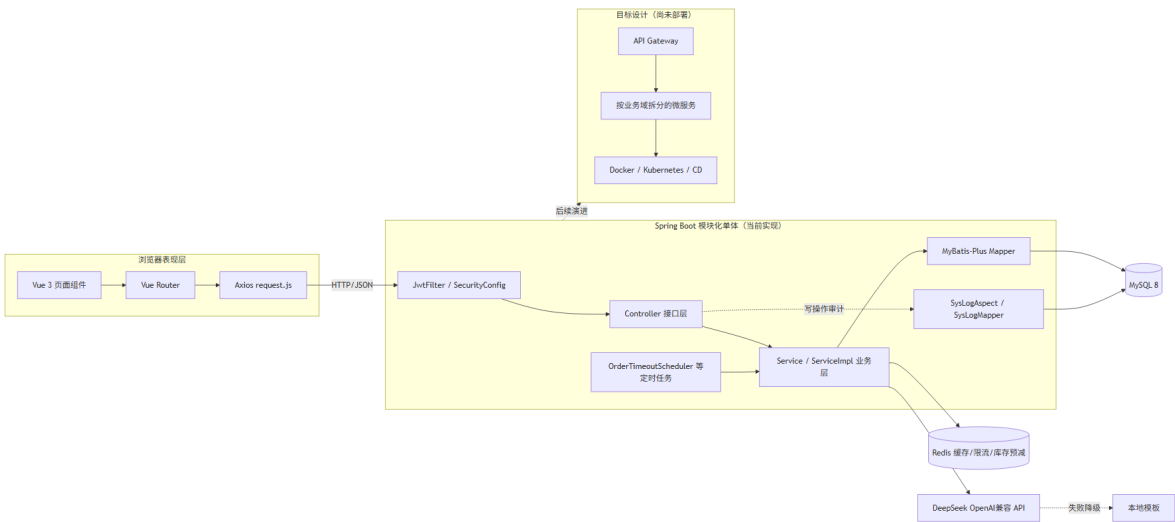
本系统的主要非功能需求如下。性能与并发指标面向课程运行环境，并在测试报告中记录验证结果。

- 1. 可用性：系统在课程验收环境中能够正常启动、正常访问，关键功能流程可完整演示；
- 2. 性能：常见查询操作和核心业务接口应具备良好的响应速度；
- 3. 稳定性：在小并发条件下，库存扣减、订单状态更新和消息通知保持正确；
- 4. 安全性：支持 JWT 认证、RBAC 授权、参数校验、敏感词过滤和统一异常处理；
- 5. 可维护性：采用分层架构、模块化组织和统一编码规范，便于多人协作和后续扩展；
- 6. 可测试性：各接口边界清晰，支持单元测试、集成测试、黑盒测试和压力测试；
- 7. 可扩展性：允许后续新增更多旅游产品、推荐逻辑、外部服务和多端适配；
- 8. 可解释性：AI 模块和核心业务规则具备明确说明与留痕能力；
- 9. 可观测性：支持关键业务数据、系统错误和访问指标的采集与可视化展示；
- 10. 常规业务接口响应时间目标控制在 500 ms 左右，AI 行程生成整体响应时间目标控制在 3 至 10 秒，并提供加载状态或流式反馈；
- 11. 系统具备处理 50 至 100 名用户同时在线查询与下单的能力，关键库存接口支持短时间内每秒数十次请求；
- 12. 当 AI 或外部接口故障时，系统需保持非 AI 主流程可访问。

2.7 系统总体架构

本系统采用前后端分离的分层架构，并将业务系统划分为用户端、管理端、业务服务层、智能服务层、数据持久层和基础设施层。

系统总体架构如下：



系统采用前端表现层、接口接入层、业务组件层、数据基础设施层和外部系统五层架构。用户端与管理端通过 Nginx、REST API 和 JWT/RBAC 接入身份、大交通、酒店目的地、AI 与消息、社区及后台治理组件；底层使用 MySQL、Redis、文件存储和定时任务，并通过受控适配器访问 DeepSeek API 与公共余票数据源。

系统架构分层说明如下：

- 1. 表现层：由用户端前端与管理端前端构成，负责页面交互、状态展示和数据可视化；
- 2. 接入层：负责统一路由、参数接收、结果封装、鉴权校验和异常拦截；
- 3. 业务服务层：负责交通、酒店、玩乐、社区、订单、用户、后台管理等业务规则；
- 4. 智能服务层：负责 AI 行程规划、AI 客服、消息触达和降级处理；
- 5. 持久化层：由 Mapper 和数据库共同构成，负责结构化数据读写；
- 6. 基础设施层：由 Redis、日志、定时任务、监控统计、Nginx 反向代理和第三方 AI 接口组成。

五个核心子系统的协同关系如下：

- 1. 大交通子系统与酒店子系统共享统一订单中心和库存控制策略；
- 2. AI 子系统结合交通、酒店、景点和消息相关服务的数据，为行程规划和客服问答提供站内资源参考；
- 3. 社区子系统与用户中心共享账号体系，并接受后台审核服务统一管控；
- 4. 管理后台子系统对资源库、订单流、社区内容和系统日志进行统一管理；
- 5. 可观测模块横向覆盖所有业务服务，采集订单、流量、错误和热点统计信息。

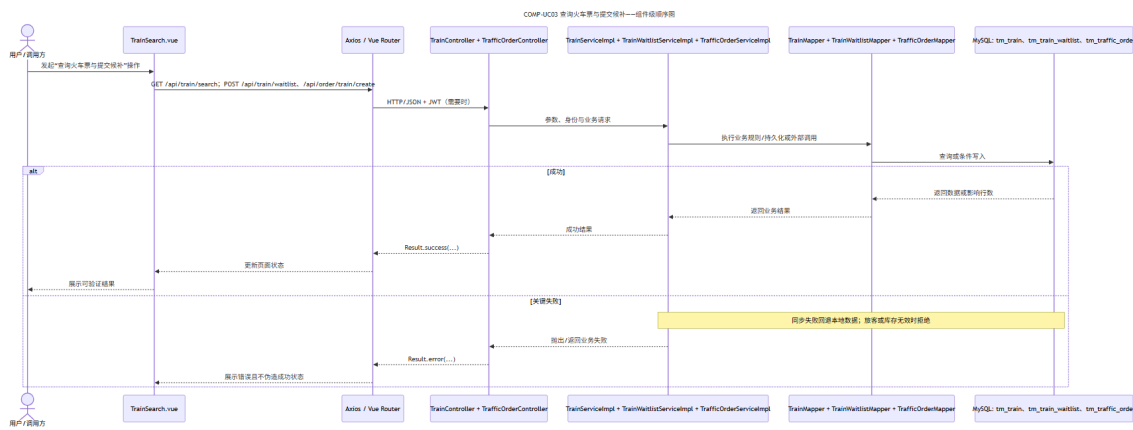
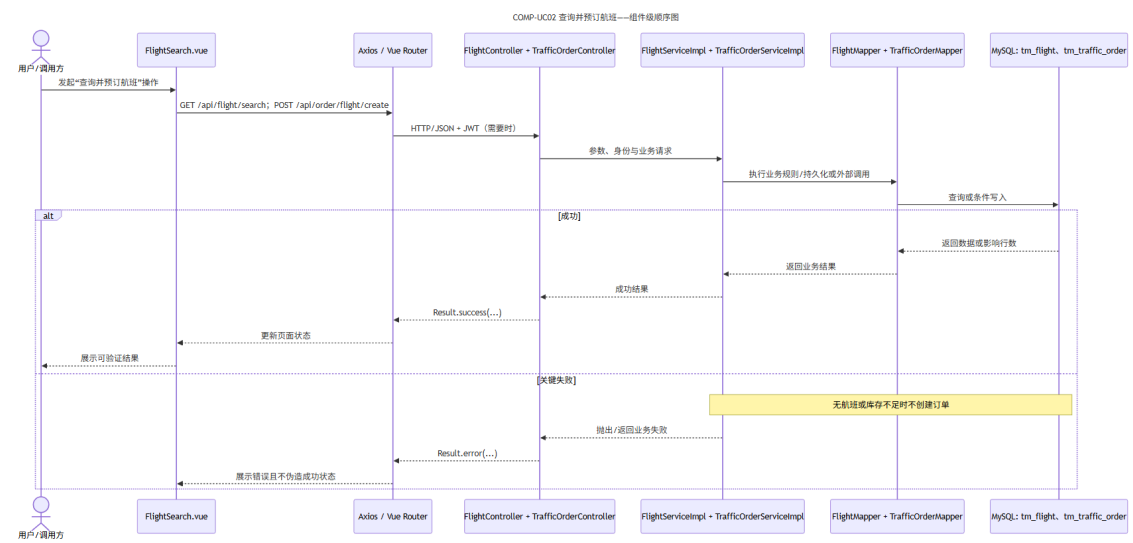
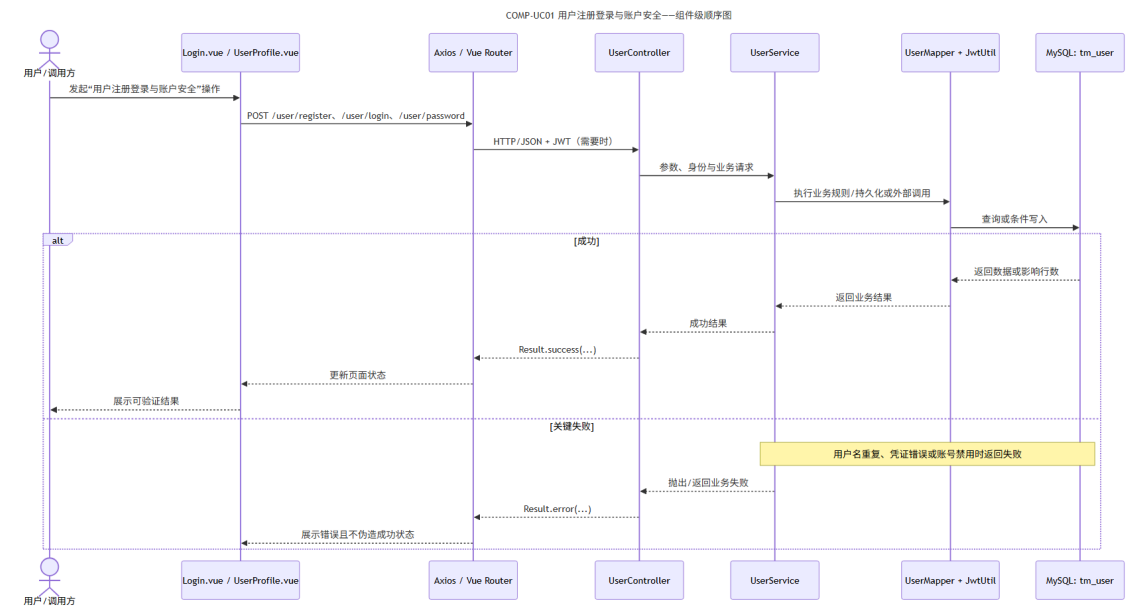
2.8逐用例组件级设计

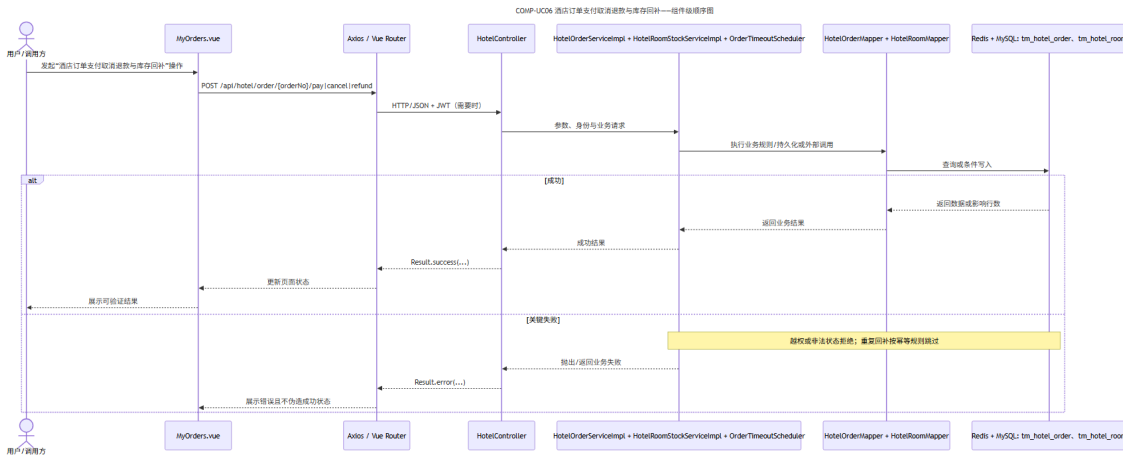
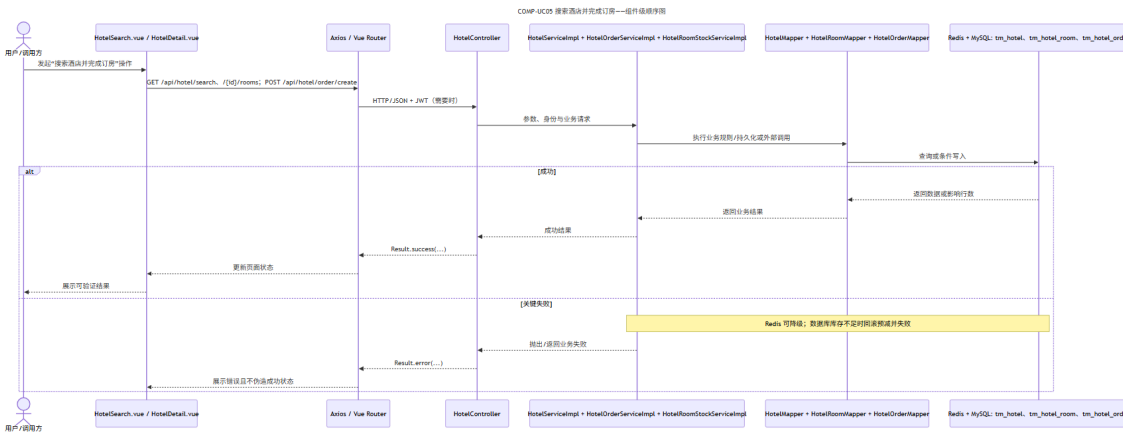
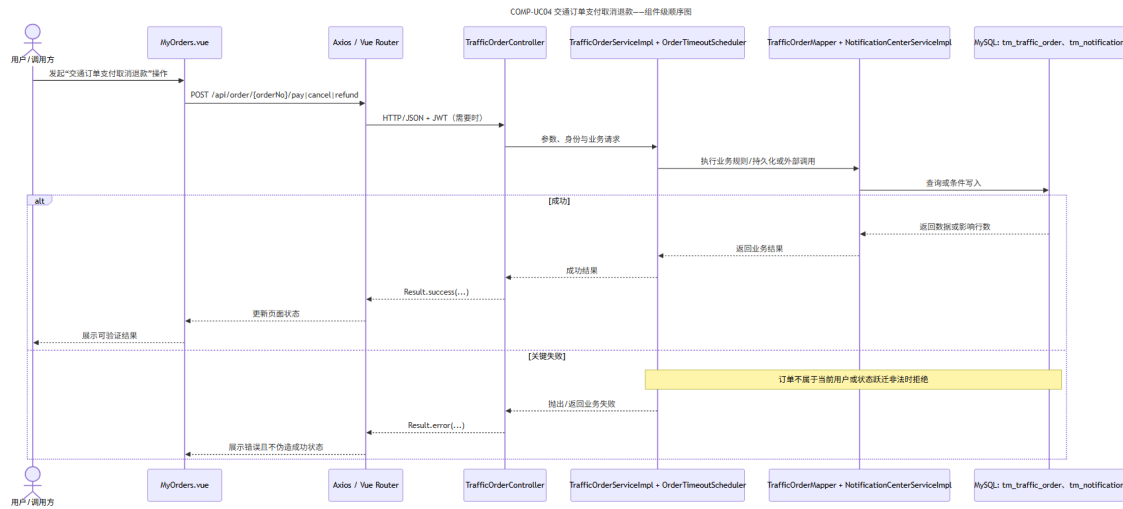
组件级模型用于把 SYS-UC01–SYS-UC19 的系统交互落实到当前仓库中的 Vue 页面、Axios 请求、Spring Boot Controller、Service、Mapper、MySQL、Redis 和外部 AI 依赖。每张图片均保留同名 Mermaid 源文件，源文件目录为 [概要设计说明/source](#)。

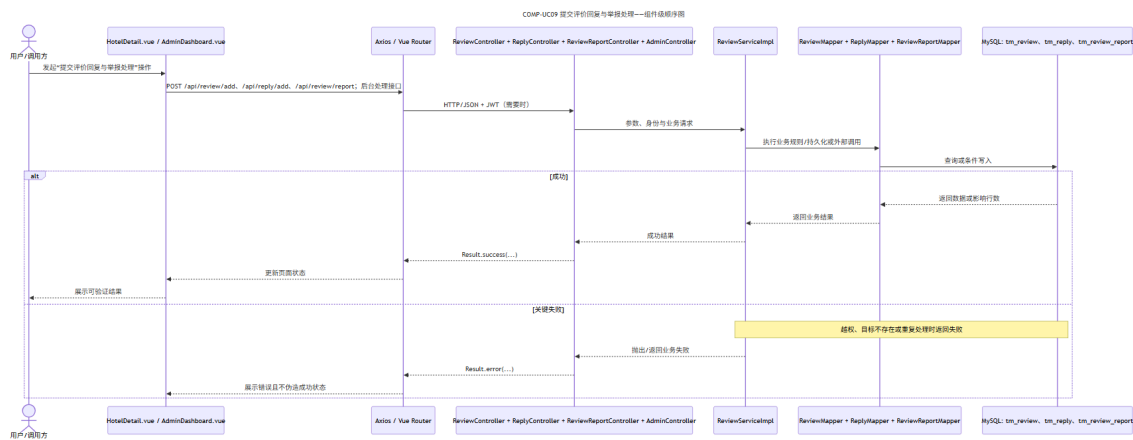
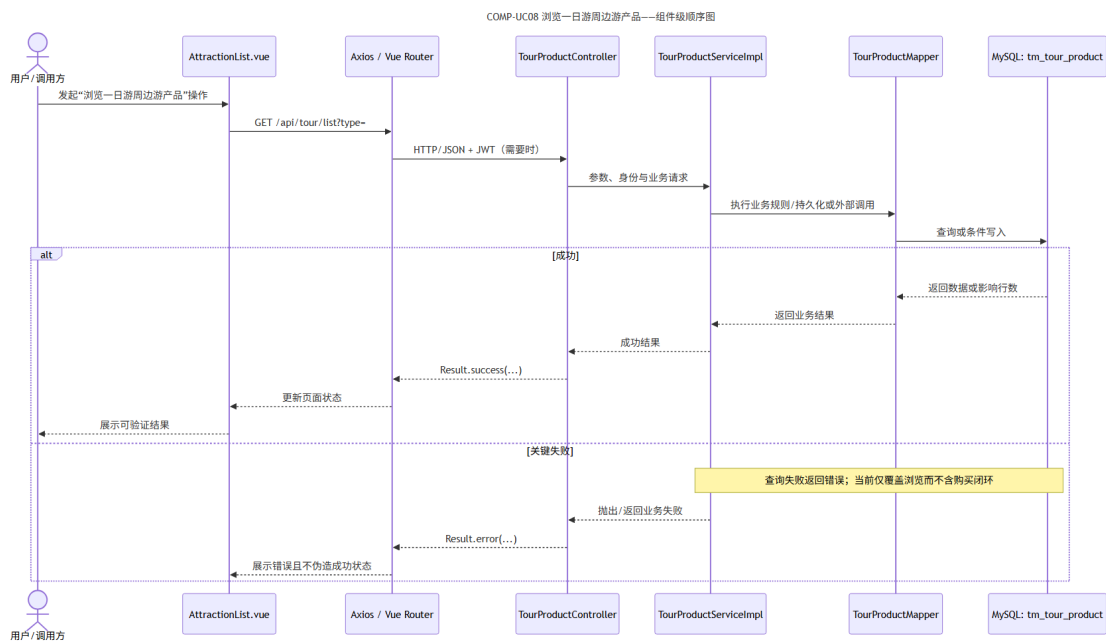
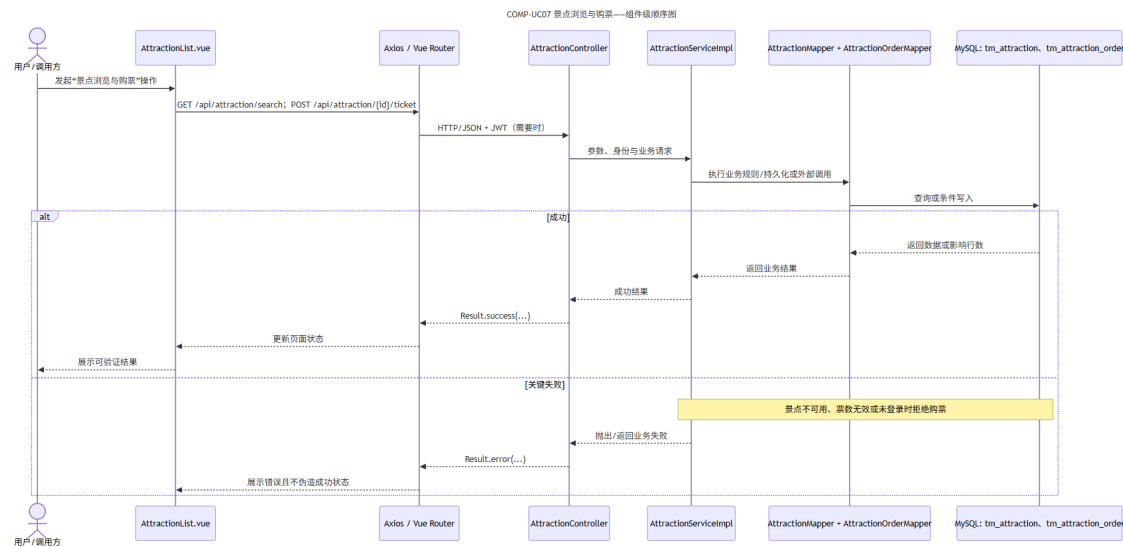
用例	组件模型	主要实现入口
UC01	COMP-UC01	Login.vue、UserController、UserService、UserMapper
UC02	COMP-UC02	FlightSearch.vue、FlightController、TrafficOrderController
UC03	COMP-UC03	TrainSearch.vue、TrainController、TrainWaitlistServiceImpl
UC04	COMP-UC04	MyOrders.vue、TrafficOrderController、TrafficOrderServiceImpl
UC05	COMP-UC05	HotelDetail.vue、HotelController、HotelOrderServiceImpl
UC06	COMP-UC06	MyOrders.vue、HotelOrderServiceImpl、HotelRoomStockServiceImpl

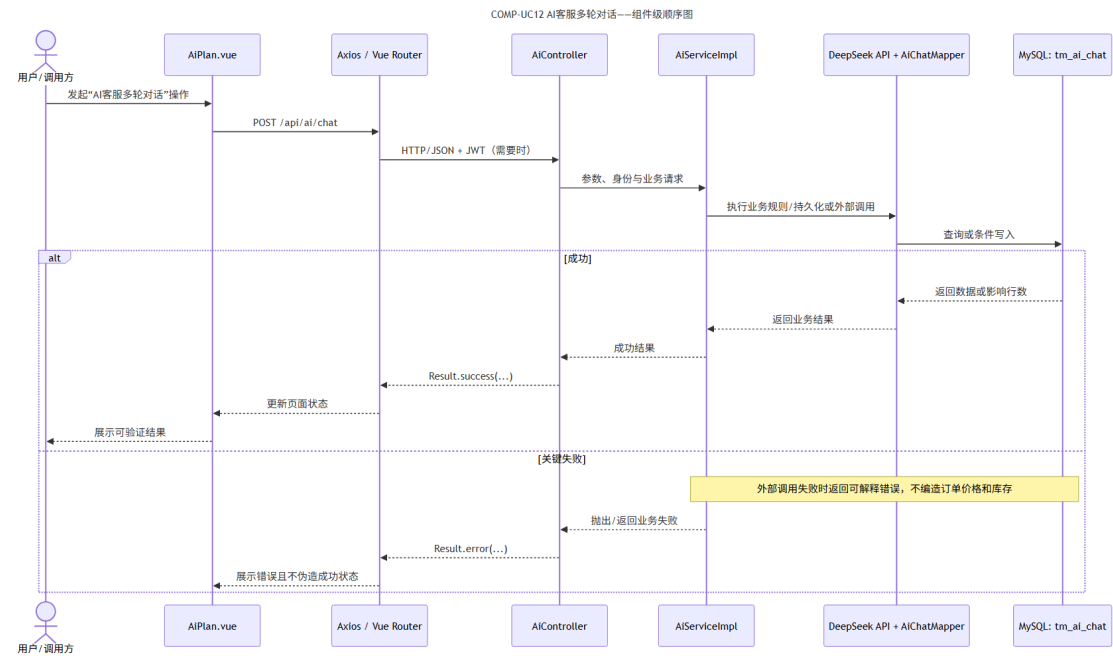
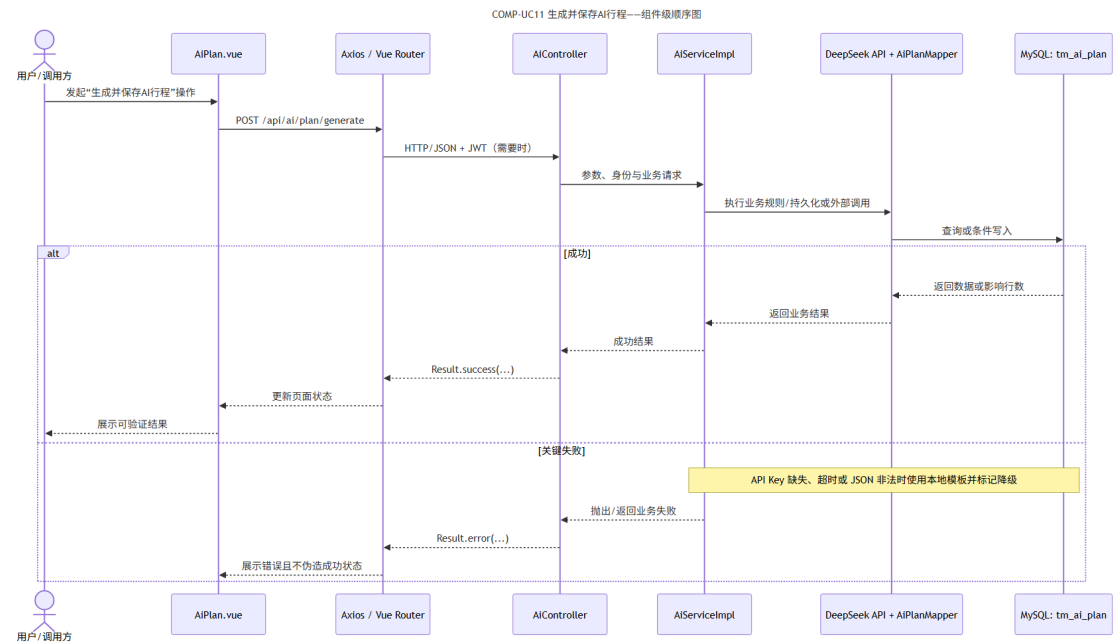
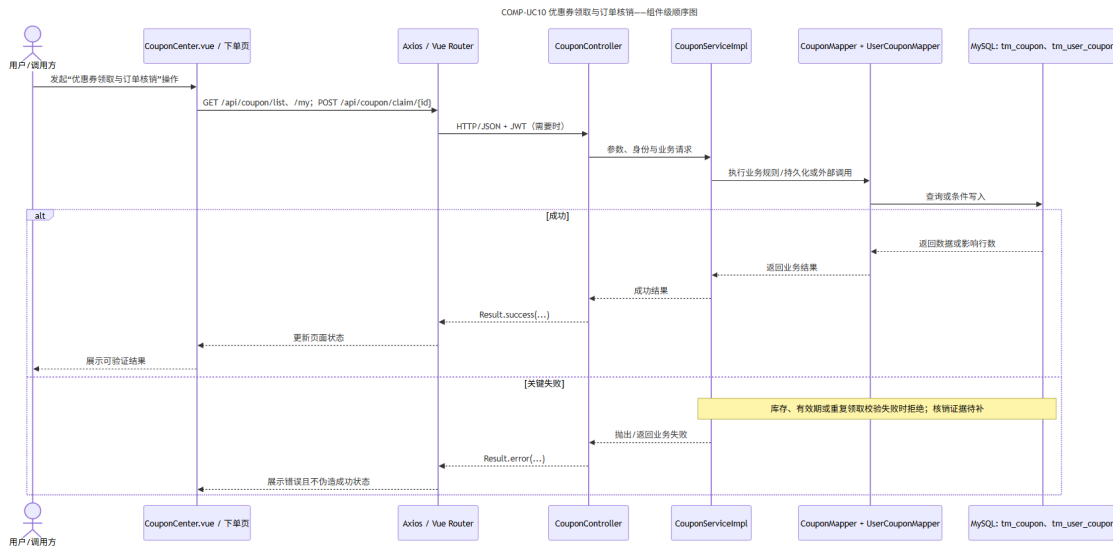
用例	组件模型	主要实现入口
UC07	<u>COMP-UC07</u>	AttractionList.vue、AttractionController、AttractionServiceImpl
UC08	<u>COMP-UC08</u>	AttractionList.vue、TourProductController、TourProductServiceImpl
UC09	<u>COMP-UC09</u>	评价/回复/举报 Controller、ReviewServiceImpl、后台处理接口
UC10	<u>COMP-UC10</u>	CouponCenter.vue、CouponController、CouponServiceImpl
UC11	<u>COMP-UC11</u>	AiPlan.vue、AiController、AiServiceImpl、DeepSeek API
UC12	<u>COMP-UC12</u>	AiPlan.vue、AiController、AiServiceImpl、AiChatMapper
UC13	<u>COMP-UC13</u>	通知/私信页面、AiController、AiServiceImpl、NotificationCenterServiceImpl、PrivateMessageController、PrivateMessageServiceImpl
UC14	<u>COMP-UC14</u>	社区页面、PostController、AdminController
UC15	<u>COMP-UC15</u>	PostDetail.vue、Like/Comment Controller 与 Service
UC16	<u>COMP-UC16</u>	交通查询页、PassengerController、PassengerServiceImpl
UC17	<u>COMP-UC17</u>	UserProfile.vue、UserProfile/Follow Controller
UC18	<u>COMP-UC18</u>	AdminDashboard.vue、SecurityConfig、AdminController
UC19	<u>COMP-UC19</u>	AdminDashboard.vue、审核/举报/敏感词/日志组件

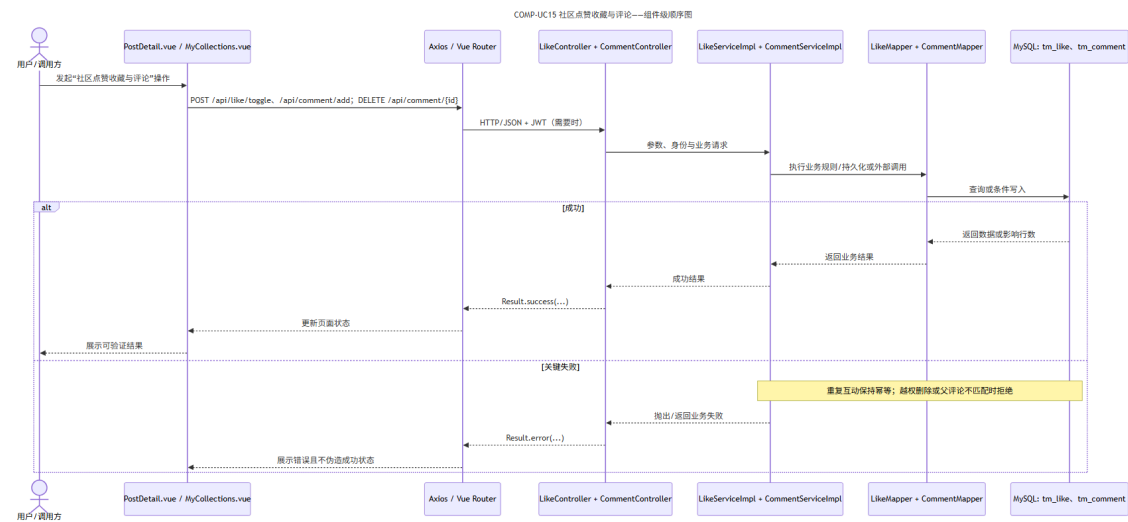
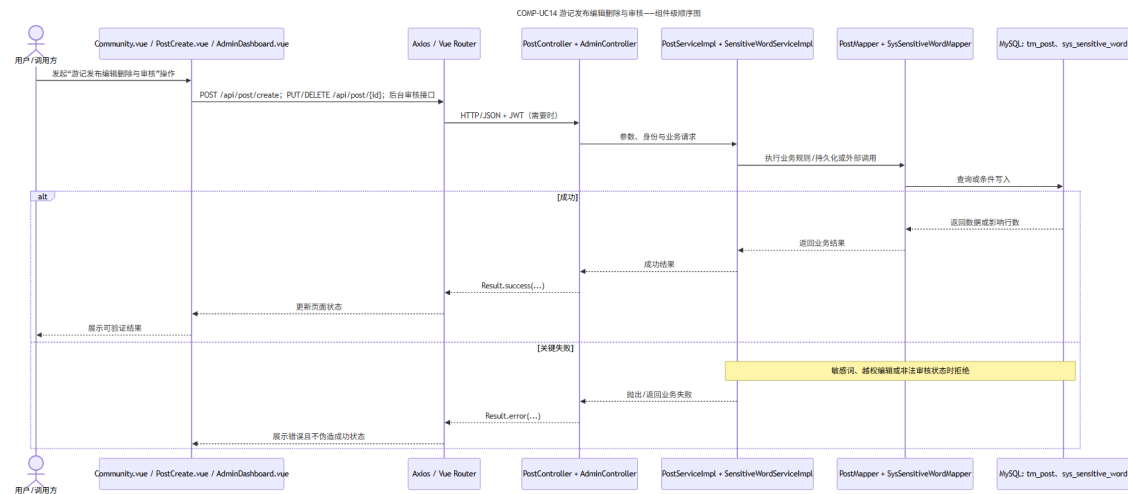
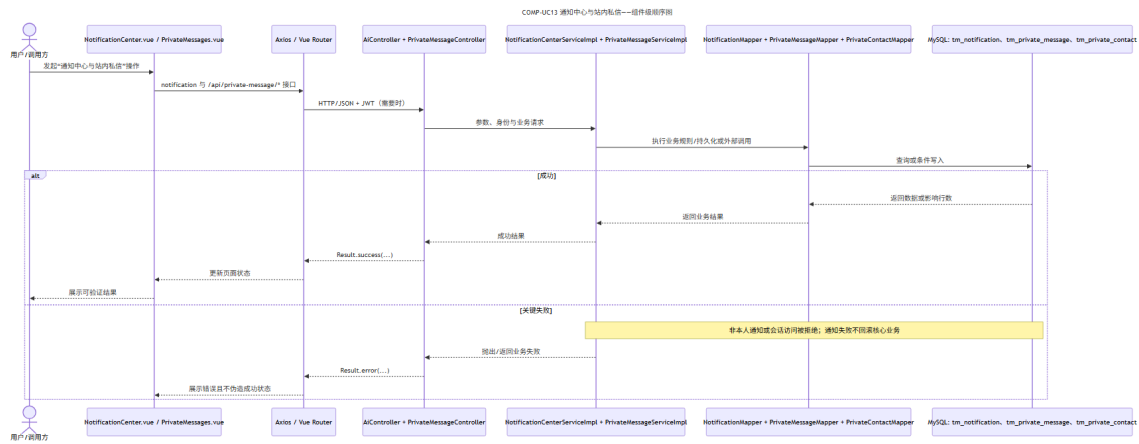
以下逐图引用用于文档验收和离线查看：

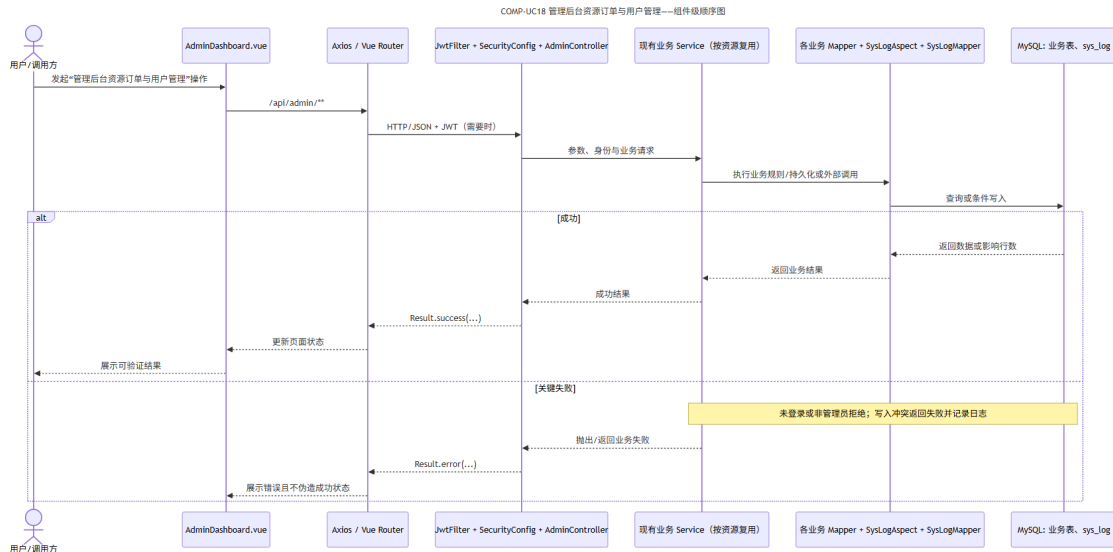
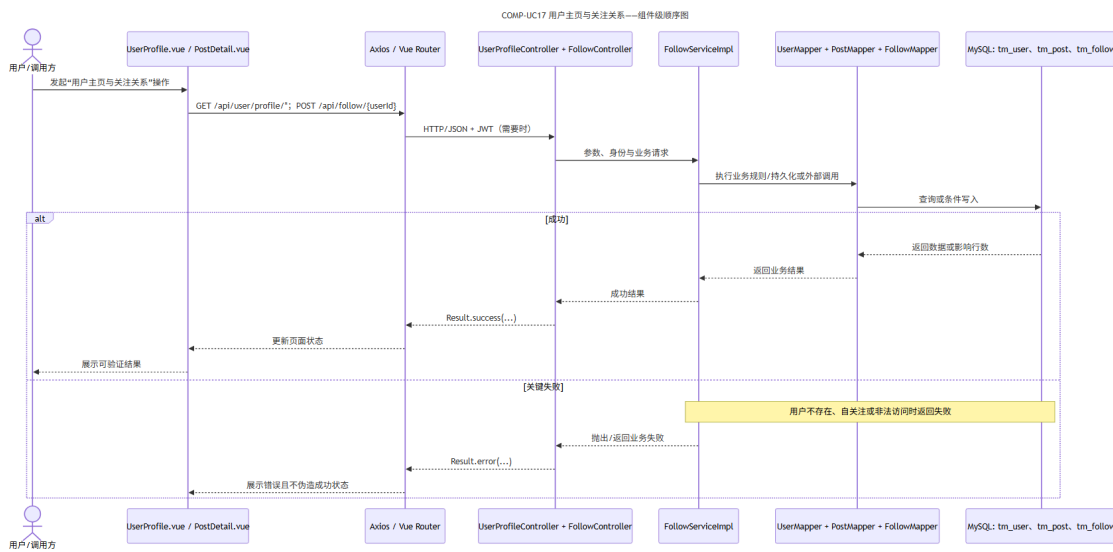
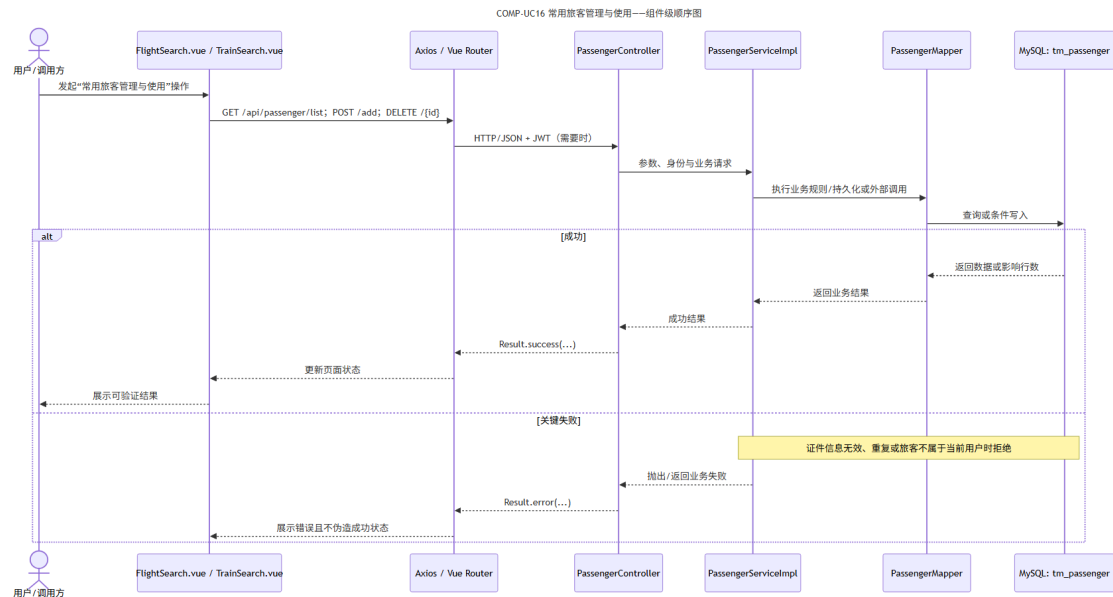


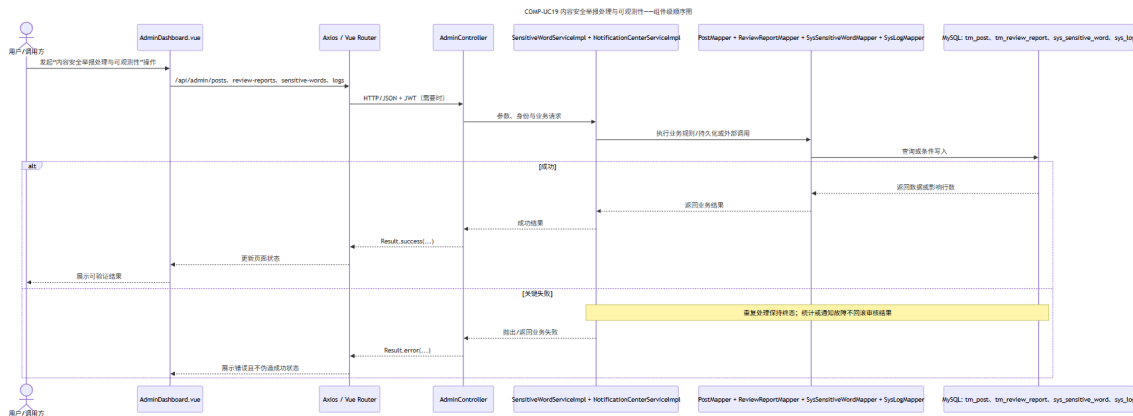












3 系统接口设计

3.1 接口设计原则

系统接口遵循以下设计原则：

- 1. 对外统一提供 RESTful 风格接口；
- 2. 接口数据格式统一为 JSON；
- 3. 统一返回结构采用 `Result` 封装；
- 4. 认证接口、业务接口、管理接口和 AI 接口按职责分层管理；
- 5. 关键接口支持参数校验、权限控制和统一异常返回；
- 6. 内部服务间主要采用同步调用，辅以定时任务处理订单超时、库存回滚和帖子 AI 审核等延时场景。

3.2 用户接口

面向普通用户和管理员的主要界面如下：

1. 首页与综合搜索页

提供机票、火车票、酒店、景点、本地玩乐等统一入口，支持 Tab 搜索、推荐位展示和优惠信息查看。

2. AI 规划页

提供预算、天数、偏好、人群等输入表单，支持生成并保存结构化行程、展示每日路线与预算估算，并关联站内旅游资源。

3. 资源详情页

提供航班、车次、酒店、景点的详细信息展示，包括价格、规则、设施、评价和库存状态。

4. 下单与订单页

支持填写旅客或住客信息、提交订单、模拟支付、取消订单、退款申请、查看订单状态与订单凭证。

5. 消息与行程页

支持消息分类查看、未读红点、AI 行程列表、订单提醒和审核/退款通知。

6. 社区发现页与发布页

支持瀑布流浏览、话题聚合、图文发布、目的地标记、点赞评论、关注用户和分享动态。

7. 用户中心页

支持注册登录、个人资料编辑、我的发布、我的收藏、账户安全和身份信息管理。

8. 管理后台页

支持左侧菜单导航、资源管理、内容审核、订单与退款处理、用户管理、角色权限控制和监控大屏。

本系统不涉及专用硬件控制面板，所有交互均通过浏览器界面完成。

3.3 外部与支撑接口

本系统与外部软件、支撑软件和系统内部模块的接口关系如下：

- 1. 浏览器接口：用户和管理员通过浏览器访问用户端和管理端页面；
- 2. Web 接口：前端通过 Axios 调用后端 HTTP 接口；
- 3. 数据库接口：后端通过 MyBatis-Plus 操作 MySQL；
- 4. 缓存接口：后端通过 Redis 处理酒店房型库存预减和接口限流计数；
- 5. 鉴权接口：Spring Security、JWT 过滤器和 RBAC 规则共同完成权限控制；
- 6. AI 接口：智能服务层通过 OpenAI 兼容协议调用 DeepSeek API，并在失败时使用本地降级模板；
- 7. 火车公共余票同步接口：后端通过 Java HttpClient 访问公共余票数据源，并在访问失败时回退到本地数据；
- 8. 文件接口：用于社区图片、头像、封面和凭证相关图片上传下载；
- 9. 网关接口：开发环境由 Vite 代理 /api 和 /user 到后端，部署时可通过 Nginx 提供静态资源托管、Gzip 压缩、反向代理和跨域访问支持；
- 10. 测试工具接口：通过 Playwright、Postman、Apifox 和 JMeter 完成端到端测试、接口联调与压力验证。

系统主要接口分组如下：

接口分类	代表接口	说明
用户认证接口	注册、登录、退出、获取个人信息	提供统一身份入口
交通资源接口	航班查询、车次查询、中转推荐、价格趋势	支撑大交通业务

接口分类	代表接口	说明
酒店与玩乐接口	酒店搜索、房型查询、景点预订、评价列表	支撑住宿与目的地业务
订单接口	创建订单、支付订单、取消订单、退款申请、订单列表	支撑统一订单中心
AI 接口	行程生成、AI 客服对话、行程保存、降级模板	支撑智能能力
消息接口	站内信、提醒、已读管理、未读统计	支撑通知中心
社区接口	发布动态、评论、点赞、收藏、关注、推荐流	支撑内容生态
管理接口	资源管理、审核、监控、权限配置、日志查看	支撑后台运营

3.4 系统数据结构设计

3.4.1 逻辑结构设计要点

系统逻辑数据结构围绕“用户、资源、订单、行程、社区、消息、权限、监控”八类核心对象展开。

1. 用户类数据结构

以 User 为核心，通过其资料、角色、密码和状态字段描述账号身份、个人信息、访问权限和账户安全。

2. 交通类数据结构

包括 Flight、Train、Passenger、PriceHistory、TrafficOrder，用于描述机票、火车票、常用旅客、价格波动和交通订单。

3. 酒店与目的地类数据结构

包括 Hotel、HotelRoom、HotelOrder、Attraction、AttractionOrder、TourProduct、TourProductStep、Review、Reply、ReviewReport、MediaAsset，用于描述住宿资源、本地玩乐资源、门票订单、媒体来源和评价信息。

4. AI 与行程类数据结构

包括 AiPlan、AiChat，用于描述 AI 行程生成结果和多轮对话历史。

5. 社区类数据结构

包括 Post、Comment、Like、Follow，用于描述内容发布、评论、点赞、收藏和关注关系。

6. 消息通知类数据结构

包括 Notification、PrivateMessage、PrivateContact，用于描述系统提醒、站内私信和会话联系人。

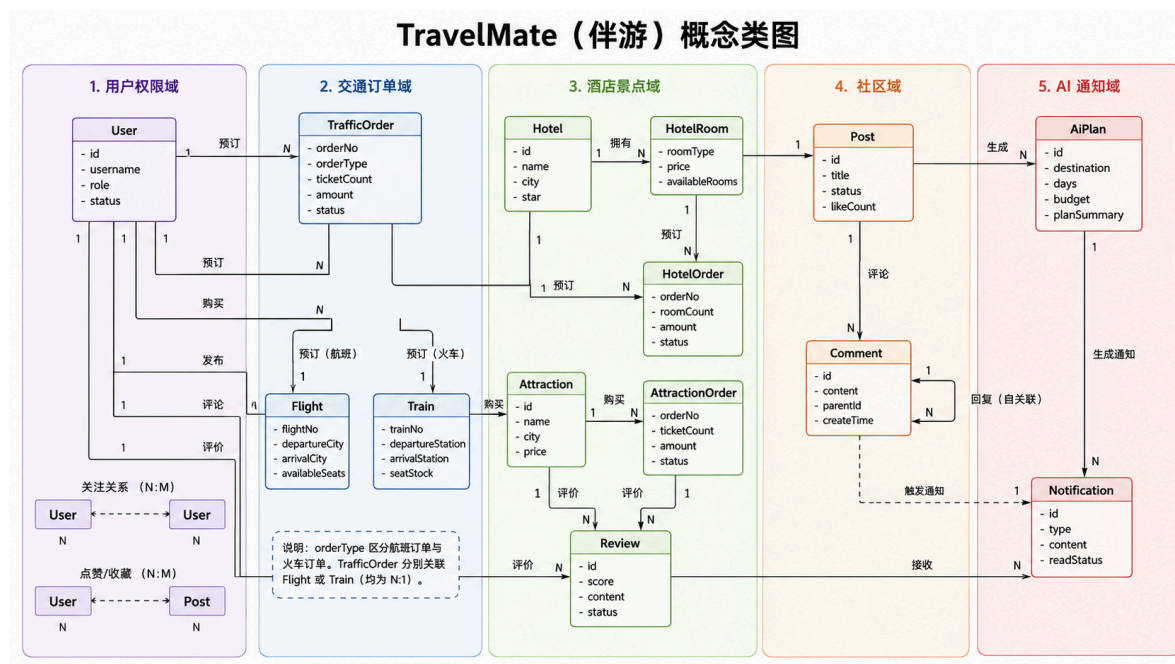
7. 后台与权限类数据结构

包括 User.role、SysLog、SysSensitiveWord、ReviewReport、Reply，用于描述后台权限、日志、敏感词、举报和回复能力。

8. 可观测类数据结构

包括 SysLog 以及由订单、用户、帖子、优惠券等业务表聚合得到的统计指标，用于描述系统日志和运营数据。

核心实体关系如下：



数据模型以 User 为中心，关联 Passenger、各类订单、AiPlan、Post、Comment、Follow 和 Notification；资源侧包含 Flight、Train、Hotel、HotelRoom、Attraction 和 TourProduct；权益与治理侧包含 Coupon、UserCoupon、Review、Reply、ReviewReport、SysSensitiveWord 和 SysLog。实体关系、基数和订单状态语义与需求规格说明书中的概念类图保持一致。

3.4.2 物理结构设计要点

系统物理结构采用 MySQL 关系型数据库为主，Redis 作为缓存和状态辅助存储，设计要点如下：

1. 业务表统一采用 `tm_` 前缀，系统日志和敏感词表采用 `sys_` 前缀；
2. 主键统一采用 BIGINT 或业务唯一编号；
3. 公共时间字段统一为 `create_time`、`update_time` 等可语义化命名；
4. 逻辑删除字段统一为 `deleted`；
5. 用户名、手机号、订单号、资源编号等关键字段建立唯一约束；

6. 高频查询条件字段建立索引，如 user_id、order_no、hotel_id、post_id、create_time；
7. 订单、支付和退款等金额字段使用 DECIMAL / BigDecimal 存储人民币元金额，避免浮点精度误差；
8. 大文本内容如社区正文、AI 结果、审核说明使用 TEXT 类型；
9. Redis 主要保存酒店房型库存预减值和接口限流计数；
10. 文件资源路径保存为 URL 或对象存储引用，不直接存大文件二进制内容。

关键物理更新策略如下：

1. 酒店房型采用“Redis Lua 预减 + 数据库原子更新”的双重控制避免超卖；航班、火车票和景点门票采用数据库原子更新避免超卖；
2. 订单创建和库存预占在同一事务中处理；
3. 未支付订单在 15 分钟超时后由 OrderTimeoutScheduler 定时任务触发自动取消与库存回滚；
4. 敏感内容发布前进行审核状态标记，审核结果异步写回；
5. 统计数据采用业务表实时统计加缓存聚合的方式生成展示结果。

3.4.3 数据结构与程序关系

数据结构与程序模块的对应关系如下：

1. User 由用户中心、认证模块和后台权限模块共同使用，管理员权限通过 role 字段区分；
2. Flight、Train、TrainWaitlist、Passenger、TrafficOrder、PriceHistory、Coupon 由大交通票务与订单模块使用；
3. Hotel、HotelRoom、HotelOrder、Attraction、AttractionOrder、TourProduct、Review、Reply、ReviewReport 由酒店与目的地服务模块使用；
4. AiPlan、AiChat 由 AI 规划与 AI 客服模块使用，Notification、PrivateMessage、PrivateContact 由消息模块使用；
5. Post、Comment、Like、Follow 由社区与社交关系模块使用；
6. SysSensitiveWord、ReviewReport、SysLog 由后台审核与安全模块使用；
7. SysLog 及各业务订单/资源表由可观测与统计模块使用；
8. DTO、VO 和统一 Result 封装由各控制器和服务模块共同使用。

4 系统数据库设计

本系统数据库采用 MySQL 8.0，数据库按业务域划分为以下几类核心表：

1. 用户与权限域

包括 tm_user、tm_follow。管理员权限通过 tm_user.role 字段与 Spring Security 角色映射实现。

2. 大交通域

包括 tm_flight、tm_train、tm_train_waitlist、tm_passenger、tm_traffic_order、tm_price_history、tm_coupon、tm_user_coupon。

3. 酒店与目的地域

包括 tm_hotel、tm_hotel_room、tm_hotel_order、tm_attraction、tm_attraction_order、tm_destination、tm_media_asset、tm_tour_product、tm_tour_product_step、tm_review、tm_reply、tm_review_report。

4. AI 与行程域

包括 tm_ai_plan、tm_ai_chat。

5. 社区域

包括 tm_post、tm_comment、tm_like、tm_follow。

6. 消息与通知域

包括 tm_notification、tm_private_message、tm_private_contact。

7. 后台与安全域

包括 sys_sensitive_word、sys_log，以及业务审核相关的 tm_post、tm_review_report。

8. 统计与可观测域

系统不单独建立统计快照表，后台仪表盘通过用户、订单、帖子、举报、优惠券和 sys_log 等表实时聚合生成。

数据库设计遵循以下原则：

1. 以统一用户主键和统一订单编号体系保证跨模块关联；
2. 以资源表、订单表、消息表和社区表四类核心表构成业务骨架；
3. 以用户角色字段、审核状态字段、举报工单和日志表构成后台治理骨架；
4. 以业务表实时聚合和日志表支撑可观测与运营分析；
5. 保证表结构与模块边界清晰对应，便于多人分工和联调测试。

5 运行设计

5.1 运行模块组合

系统运行时由以下模块组合构成：

1. 用户端前端模块

负责普通用户的搜索、规划、预订、消息、社区和个人中心交互。

2. 管理端前端模块

负责管理员和运营人员的资源管理、审核处理、订单监控、权限配置和可观测大屏展示。

3. 认证与权限模块

负责登录注册、JWT 校验、角色识别和接口权限控制。

4. 大交通票务模块

负责航班、车次、价格趋势、旅客管理和交通订单处理。

5. 酒店与目的地模块

负责酒店搜索、房务预订、景点与本地玩乐预订、评价与反馈。

6. AI 智能服务模块

负责 AI 行程规划、AI 客服、多轮对话、行程保存和降级模板。

7. 社区与用户中心模块

负责 UGC 发布、瀑布流推荐、社交关系、用户中心和收藏。

8. 管理后台与审核模块

负责资源管理、内容审核、订单与退款审核、权限治理和系统配置。

9. 消息通知与可观测模块

负责站内信、提醒、日志记录、统计聚合和仪表盘数据生成。

10. 数据持久化与缓存模块

负责数据库读写、缓存、延迟任务状态和热点资源加速。

11. 运行保障与降级机制

负责在正常、维护和降级三种运行模式之间切换，并在 AI、Redis 或公共余票同步等依赖异常时执行兜底控制。

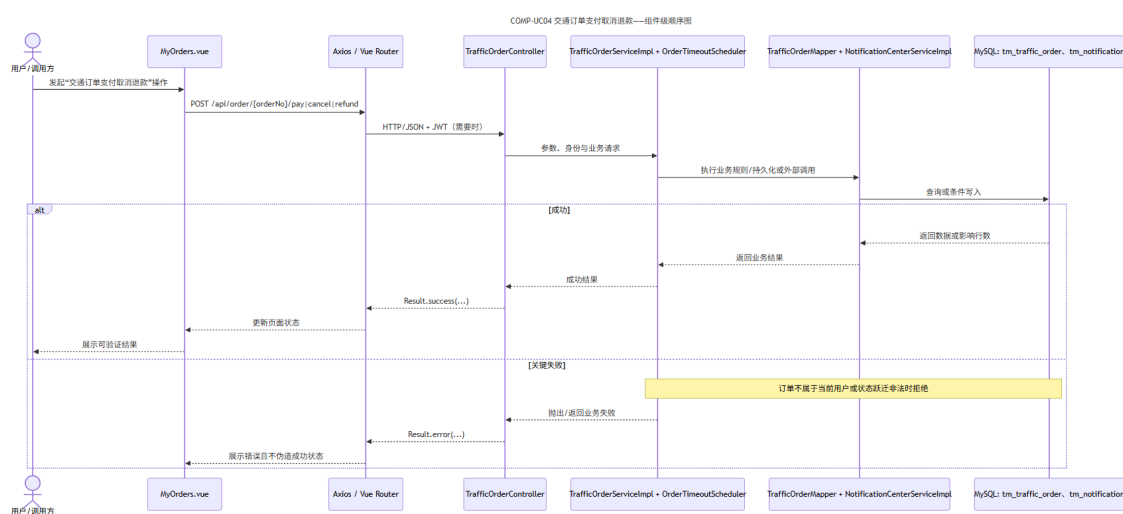
5.2 运行控制

系统运行控制遵循以下机制：

1. 前端通过统一路由进入功能页并发起 HTTP 请求；
2. 接口接入层统一完成参数校验、权限校验和结果封装；
3. 业务处理由 Service 层集中实现，复杂写操作通过事务控制；

4. 库存扣减、订单创建、消息通知和日志记录按业务优先级分别采用同步与异步组合处理；
5. 酒店下单场景优先在 Redis 完成库存预减，再由数据库原子更新完成最终一致性校验；航班、火车票和景点门票通过数据库原子更新直接校验库存；
6. AI 模块先进行上下文整理，再调用模型服务，并按结构化结果回填前端；当 AI 接口失败时，返回预设行程模板或兜底客服回答；
7. 系统支持正常运行、维护运行和降级运行三种控制模式；
8. 社区内容在发布后先进入审核状态，再按审核结果决定可见范围；
9. 管理后台通过 RBAC 控制资源访问与操作范围；
10. 可观测模块周期性汇总订单、访问、错误和热门目的地指标；
11. 超时未支付订单通过 `OrderTimeoutScheduler` 定时任务自动回滚库存与房态。

订单与通知核心运行时序如下：



用户端提交订单后，由订单 Controller 和 Service 完成身份、参数及库存校验，库存组件执行预减或原子扣减，订单 Mapper 在事务中写入订单。事务提交后，通知 Service 写入业务通知并返回订单号；库存不足或数据库写入失败时回滚并释放预占，通知失败仅记录日志，不回滚已经成功的核心订单。

5.3 运行时间

在课程项目环境下，系统运行时间目标如下：

1. 用户端和管理端启动后能够稳定访问；
2. 搜索、列表、详情等常见查询操作具备良好响应速度；
3. 下单、支付、取消、审核等关键写操作在小并发场景下保持稳定；
4. AI 行程规划和客服响应时间控制在可接受的交互范围内，并支持失败兜底；
5. 定时任务、统计任务和延时回滚任务按分钟级或秒级策略稳定运行；
6. 系统支持按需启动、停止和重启，适合本地开发、联调、测试、验收和答辩展示。

系统通过 Nginx 提供静态资源分发、Gzip 压缩和反向代理能力，前端通过路由懒加载与资源分包减少弱网环境下的加载压力。

综上，TravelMate 采用“前后端分离、统一用户体系、统一订单模型、智能服务增强、后台治理与可观测协同”的总体方案。该方案明确了大交通、酒店与目的地、AI 规划与客服、旅途社区、管理后台等模块的职责和协作边界，并通过追溯表将详细设计、代码和测试与 UC01—UC19 对应起来。

6 组件设计与逐用例组件级模型

6.1 系统组件图

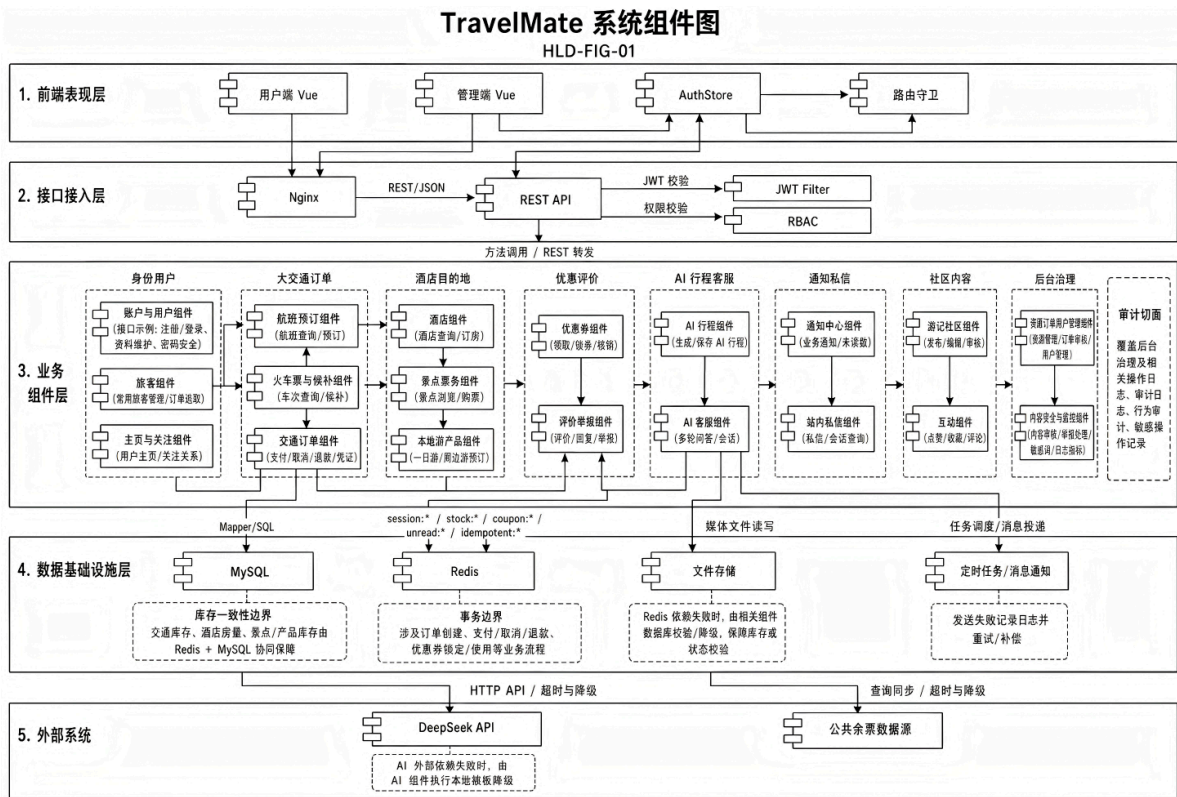


图 HLD-FIG-01 TravelMate 系统组件图

系统组件图从前端表现层、接口接入层、业务组件层、数据基础设施层和外部系统五个层次展示 TravelMate 的总体组成。用户端与管理端统一通过 Nginx 和 REST API 访问业务组件，JWT Filter 与 RBAC 负责认证授权，业务服务通过 Mapper、Redis、文件存储和定时任务完成持久化、缓存及异步处理，并对 DeepSeek API 和公共余票数据源设置降级边界。

6.2 UC01—UC19 组件级模型图

本节按 UC01—UC19 展示各业务场景在页面、Controller、Service、Mapper、数据库及外部依赖之间的组件协作关系。组件级模型承接需求规格说明书中的系统级模型，并为详细设计中的对象级交互和测试追溯提供实现边界。

HLD-FIG-02 COMP-UC01 用户注册、登录与账户安全组件级顺序图

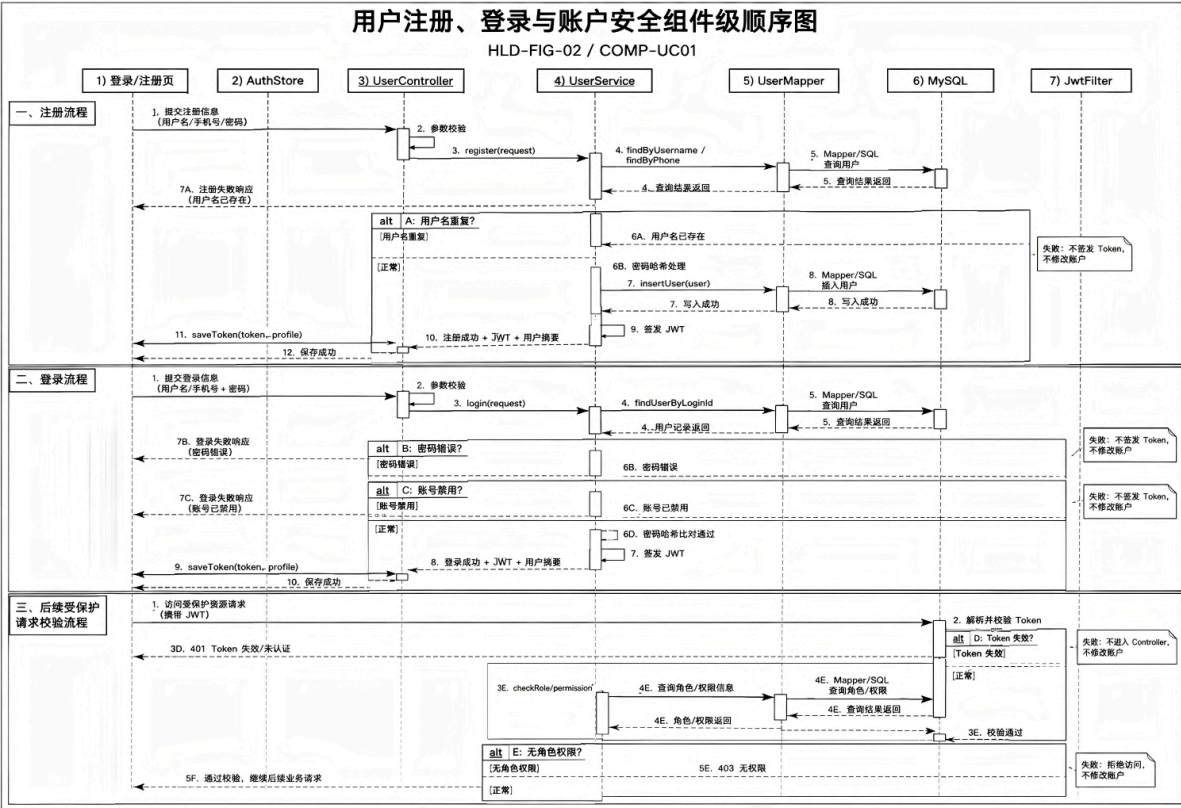


图 HLD-FIG-02 COMP-UC01 用户注册、登录与账户安全组件级顺序图

登录注册页面通过 UserController 和 UserService 完成参数校验、密码哈希处理与账户查询, UserMapper 负责持久化, 成功后由系统签发 JWT 并交由 AuthStore 保存。用户名重复、凭据错误、账号禁用或 Token 失效时, 流程在对应组件处终止且不修改账户数据。

HLD-FIG-03 COMP-UC02 查询并预订航班组件级顺序图

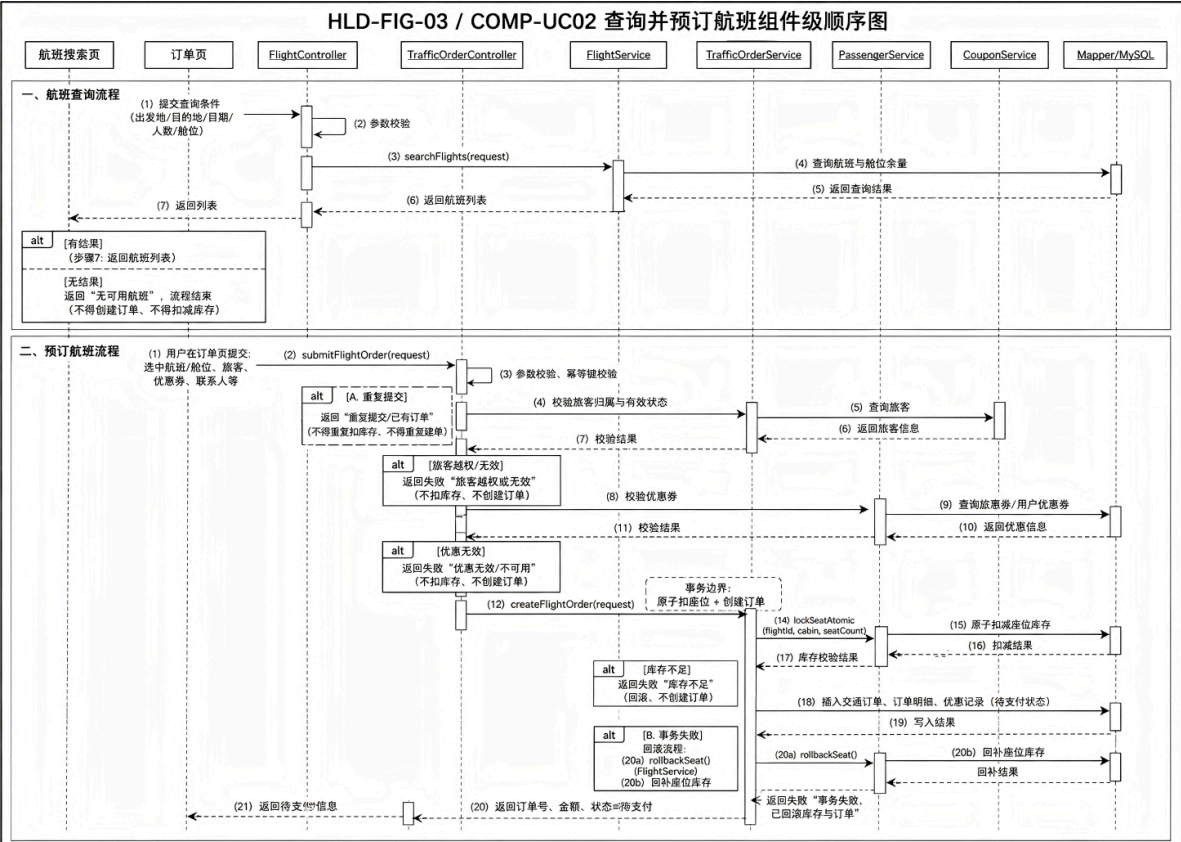


图 HLD-FIG-03 COMP-UC02 查询并预订航班组件级顺序图

航班查询由 FlightController 和 FlightService 完成，预订请求转交 TrafficOrderController 与 TrafficOrderService，并协同旅客、优惠和库存组件创建待支付订单。库存不足、旅客越权、优惠无效、重复提交或事务失败时，订单与座位库存保持一致。

HLD-FIG-04 COMP-UC03 查询并预订火车票或提交候补组件级活动图

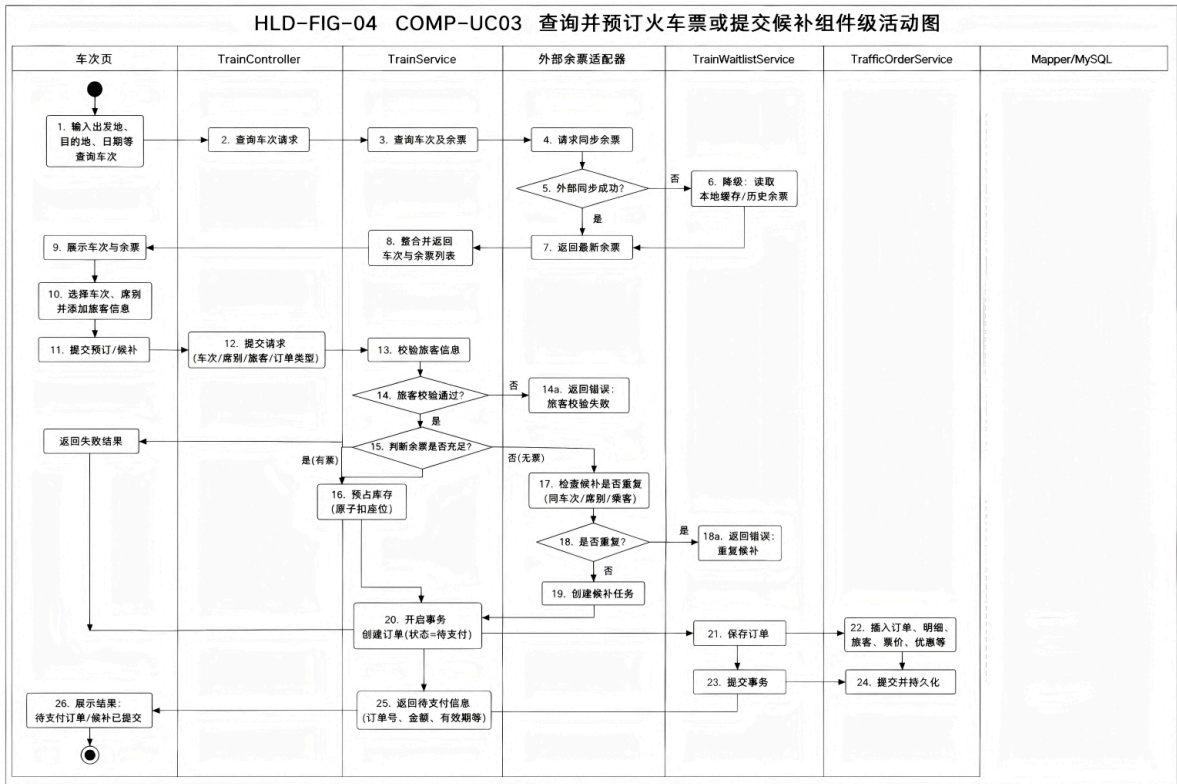


图 HLD-FIG-04 COMP-UC03 查询并预订火车票或提交候补组件级活动图

TrainService 通过外部余票适配器获取车次和余票，外部同步失败时降级读取本地数据。有票时由 TrafficOrderService 创建订单，无票时由 TrainWaitlistService 创建候补记录，旅客校验或库存竞争失败不会产生无效业务数据。

HLD-FIG-05 COMP-UC04 交通订单支付、取消、退款与凭证查看组件级状态图

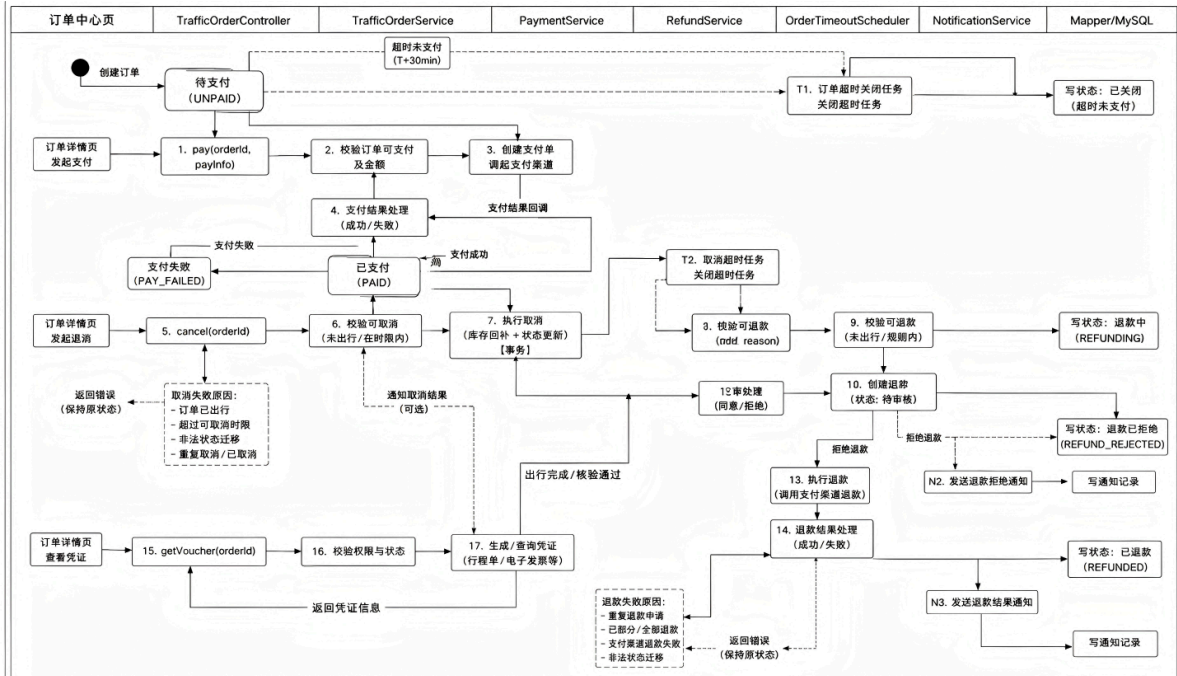


图 HLD-FIG-05 COMP-UC04 交通订单支付、取消、退款与凭证查看组件级状态图

交通订单状态由 TrafficOrderService 统一控制，PaymentService、RefundService、超时任务和通知组件分别参与支付、退款、关闭与结果通知。所有状态迁移均需校验订单归属和当前状态，取消或退款涉及的库存回补与订单更新处于受控事务中。

HLD-FIG-06 COMP-UC05 搜索酒店并完成订房组件级顺序图

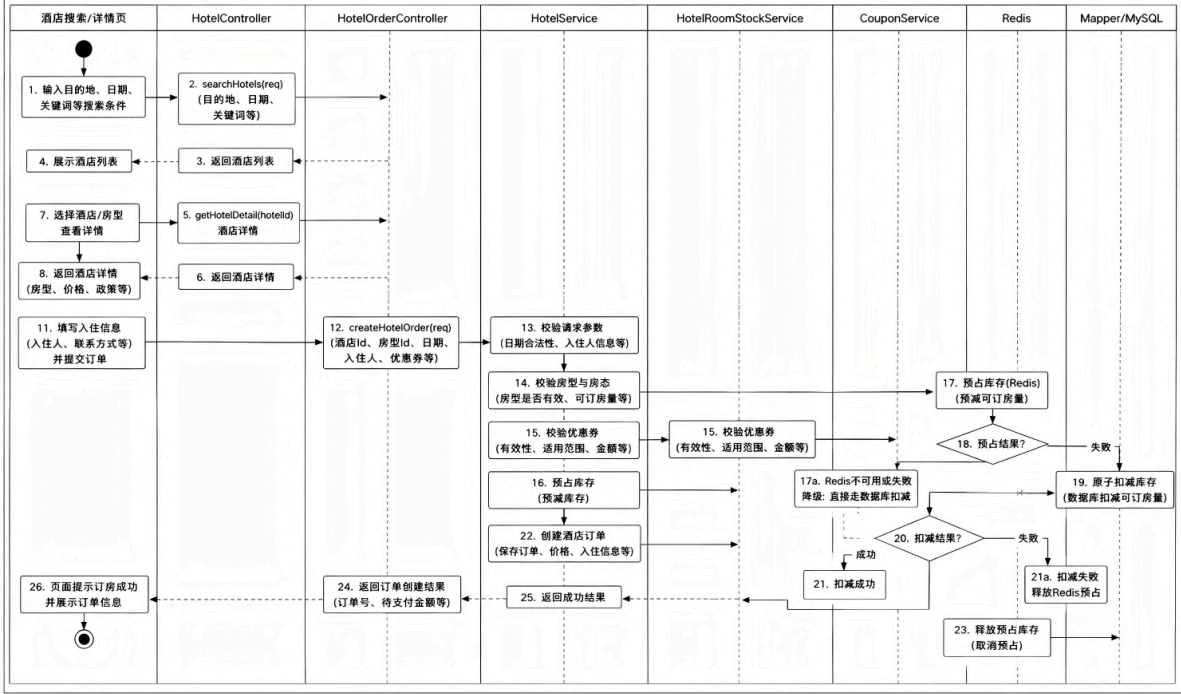


图 HLD-FIG-06 COMP-UC05 搜索酒店并完成订房组件级顺序图

HotelController 提供酒店搜索与详情查询，HotelOrderController 将订房请求交由酒店、优惠和库存服务协同处理。系统先执行 Redis 预减，再完成数据库原子扣减和订单持久化；任一步失败均释放预占并阻止脏订单写入。

HLD-FIG-07 COMP-UC06 酒店订单支付、取消、退款与库存回补组件级状态图

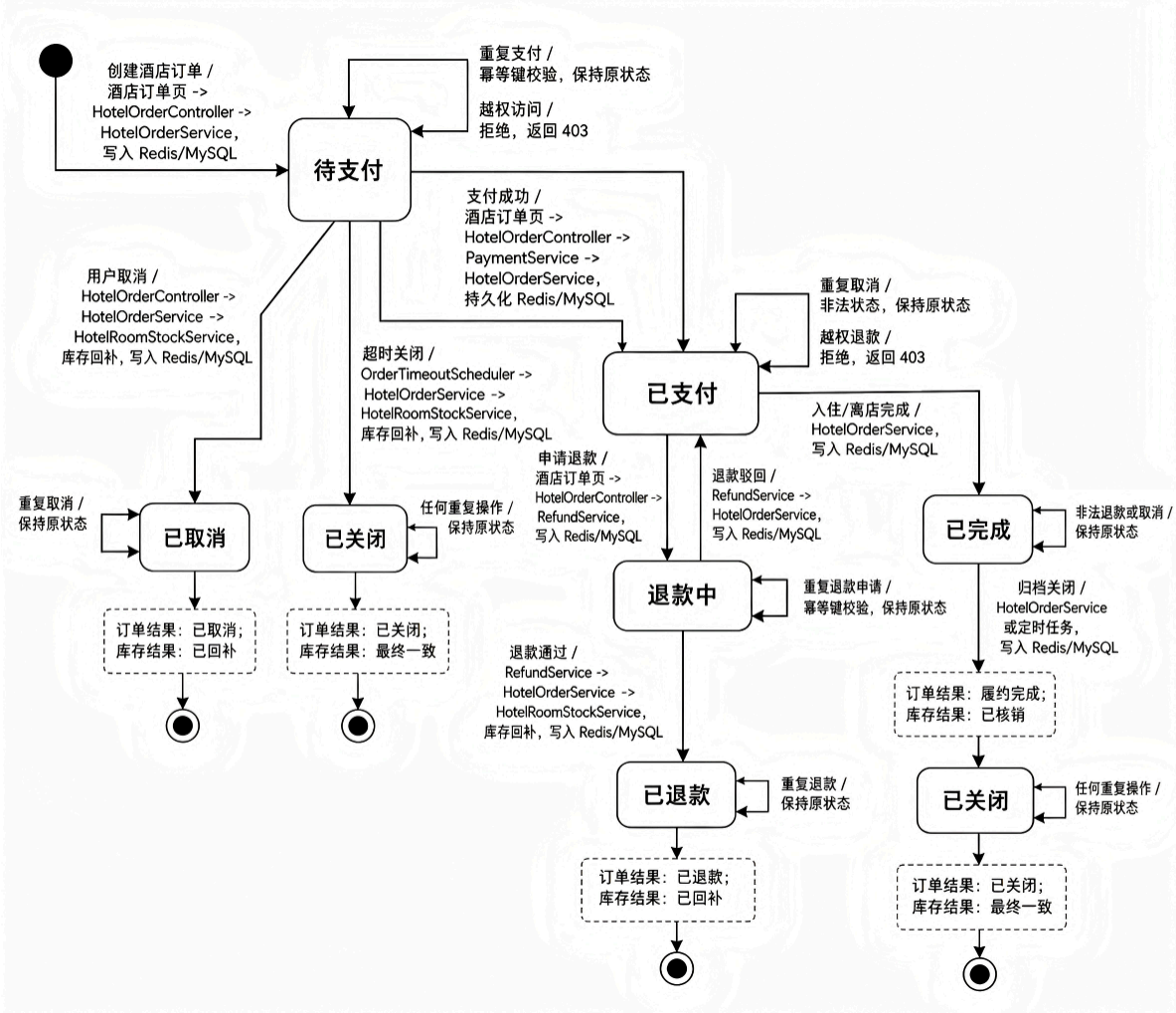


图 HLD-FIG-07 COMP-UC06 酒店订单支付、取消、退款与库存回补组件级状态图

酒店订单在待支付、已支付、退款中、已退款、已取消、已完成和已关闭等状态间迁移。HotelOrderService 负责业务状态控制，HotelRoomStockService 负责库存回补，并通过幂等键和补偿机制保证 Redis 与 MySQL 最终一致。

HLD-FIG-08 COMP-UC07 景点浏览与购票组件级顺序图

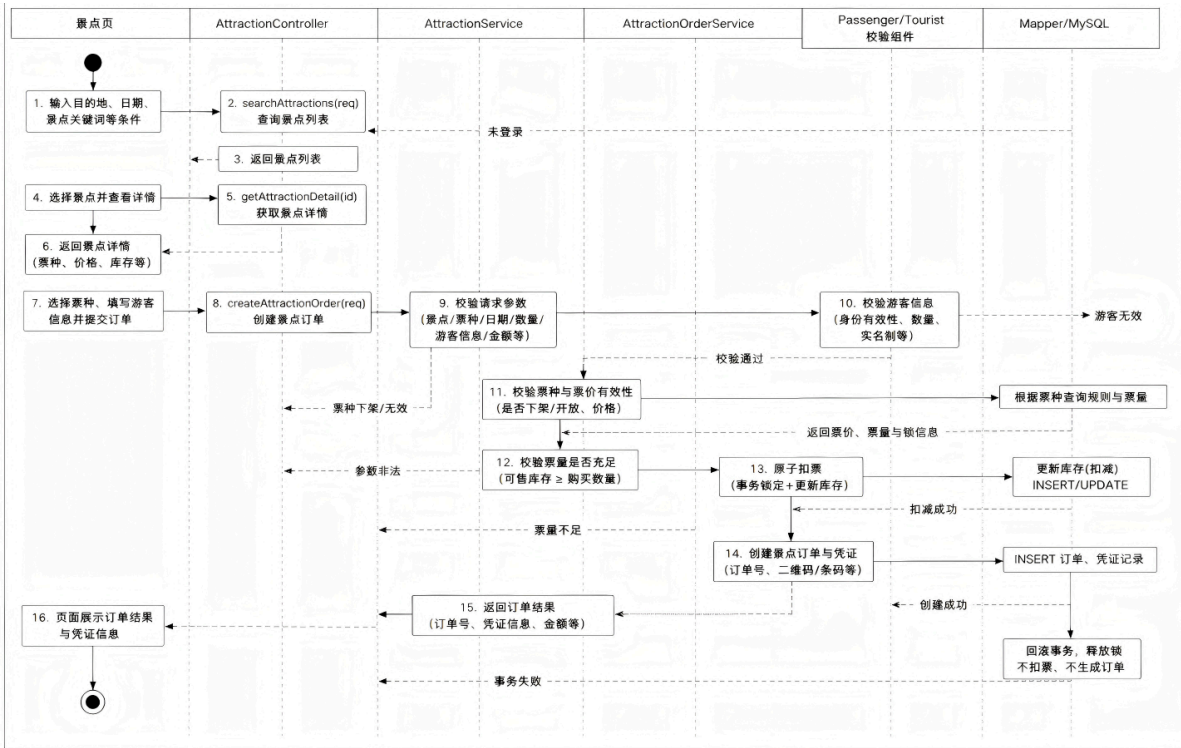


图 HLD-FIG-08 COMP-UC07 景点浏览与购票组件级顺序图

该模型展示景点页面、AttractionController、AttractionService、AttractionOrderService、游客校验组件和 Mapper/MySQL 的协作关系。系统依次完成景点查询、票种与游客校验、票量原子扣减、订单创建和电子凭证生成；未登录、票种下架、金额不一致、票量不足或事务失败时，不保留无效订单。

HLD-FIG-09 COMP-UC08 选择并预订一日游或周边游产品组件级活动图

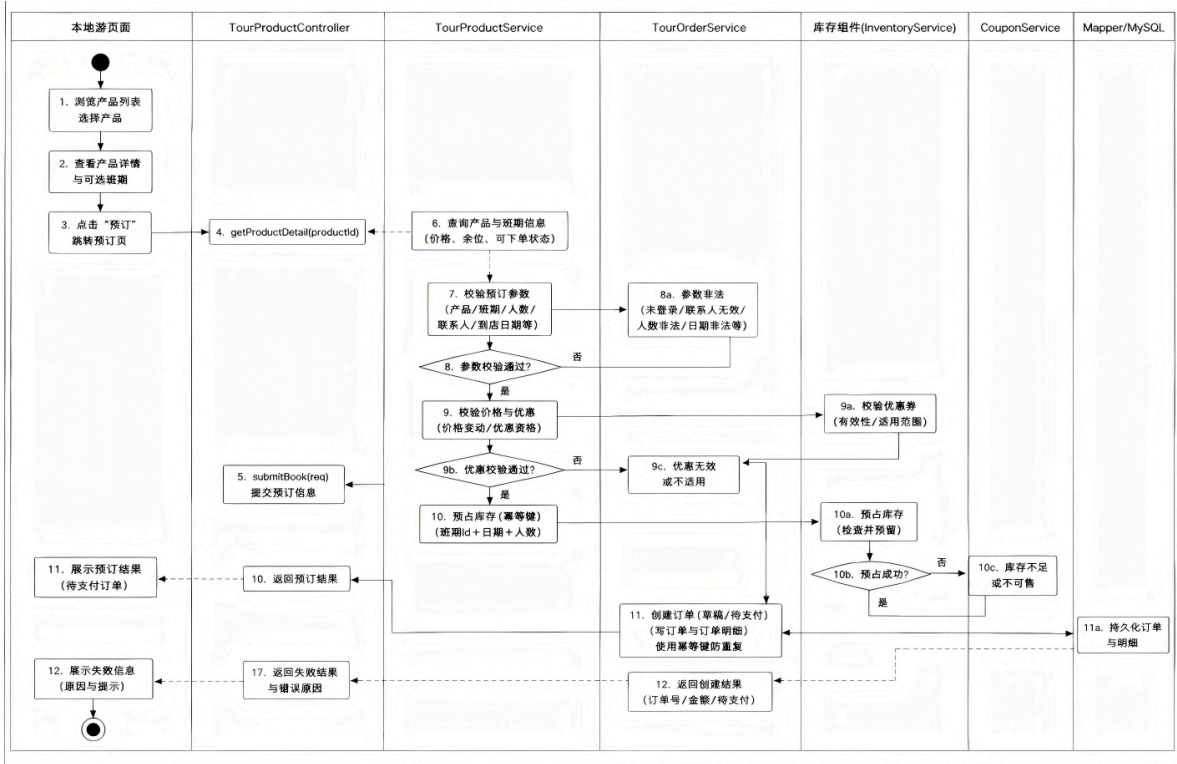


图 HLD-FIG-09 COMP-UC08 选择并预订一日游或周边游产品组件级活动图

本地游页面经 TourProductController 查询产品及班期，预订时由 TourProductService、TourOrderService、库存组件和 CouponService 完成参数、价格、优惠及库存校验。创建订单与库存扣减在事务边界内完成，失败请求不写入订单。

HLD-FIG-10 COMP-UC09 提交评价、回复评价与举报处理组件级活动图

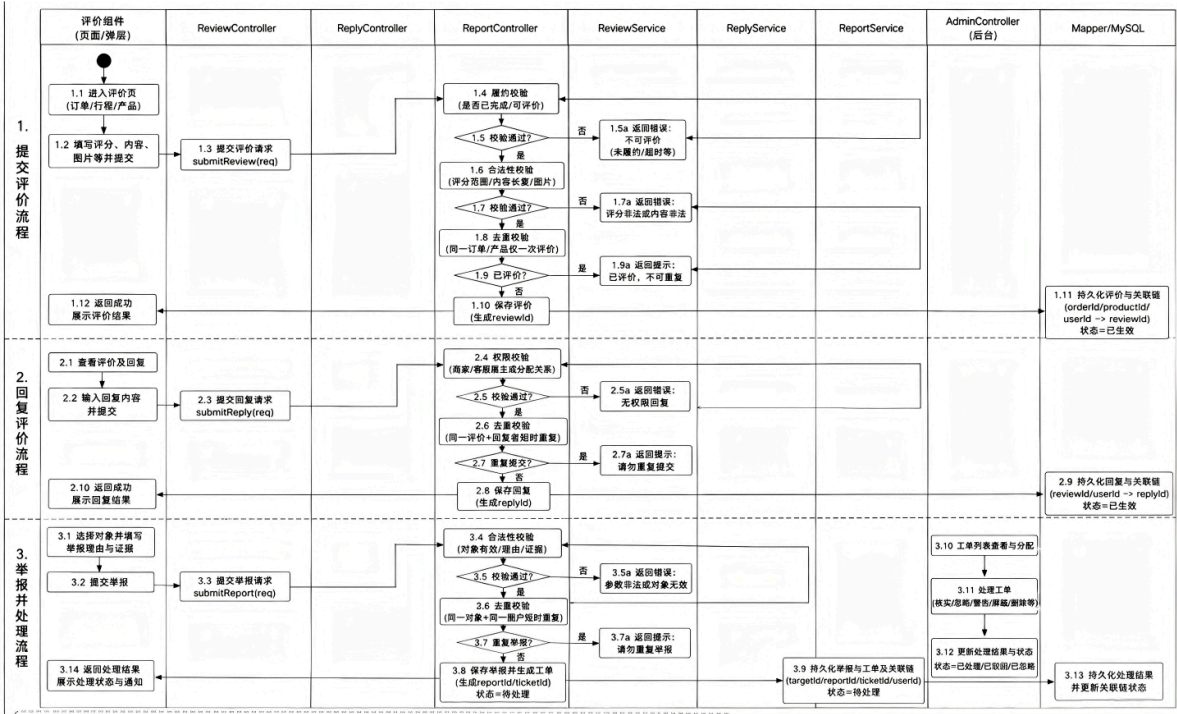


图 HLD-FIG-10 COMP-UC09 提交评价、回复评价与举报处理组件级活动图

评价、回复和举报分别由对应 Controller 与 Service 处理，并通过订单或产品关系形成统一业务链。系统校验履约资格、回复权限及重复操作，举报成功后生成工单，由后台处理并持久化最终结果。

HLD-FIG-11 COMP-UC10 优惠券领取与订单核销组件级状态图

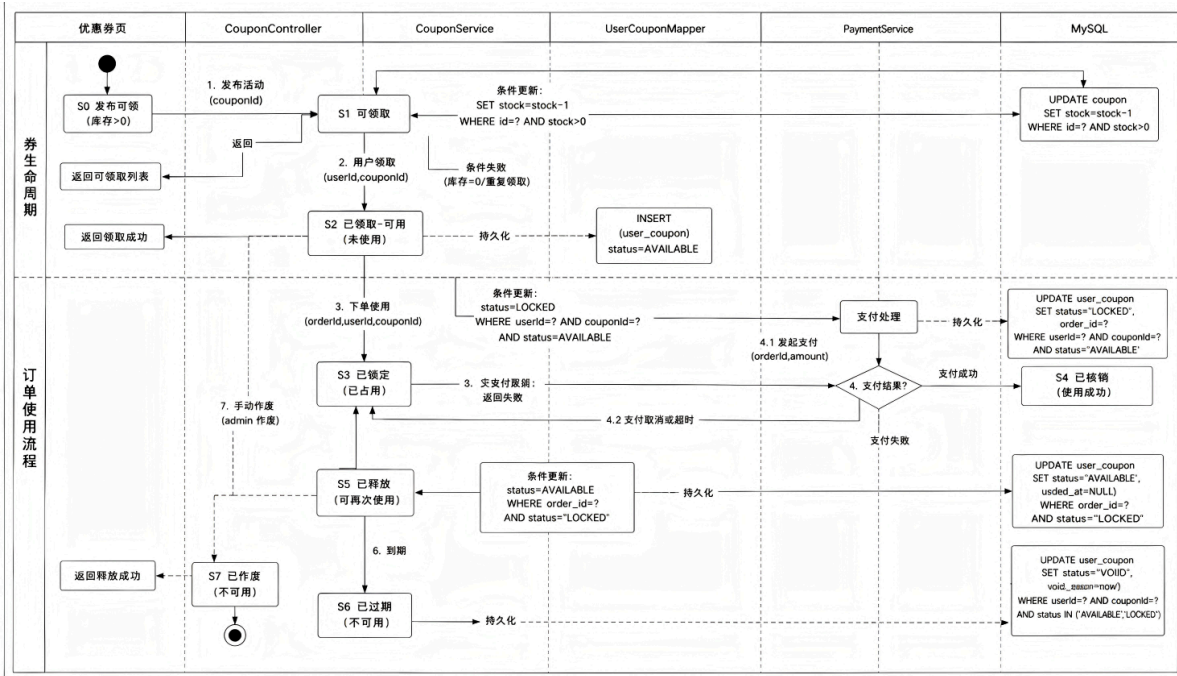


图 HLD-FIG-11 COMP-UC10 优惠券领取与订单核销组件级状态图

CouponService 管理优惠券发布与领取，UserCouponMapper 保存用户优惠券状态，PaymentService 在订单支付过程中完成锁定、核销或释放。条件更新和唯一约束保证优惠券库存、领取关系和核销状态的一次性。

HLD-FIG-12 COMP-UC11 生成并保存 AI 行程组件级顺序图

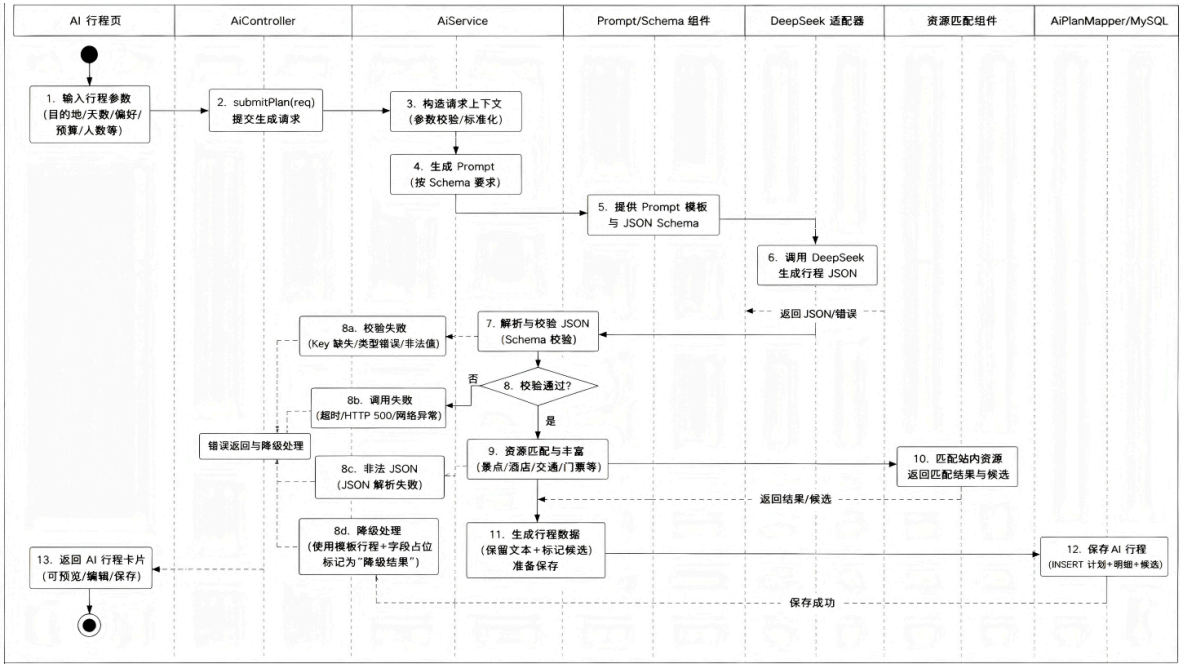


图 HLD-FIG-12 COMP-UC11 生成并保存 AI 行程组件级顺序图

AI 行程页面通过 AiController 调用 AiService，依次完成 Prompt 构造、Schema 校验、DeepSeek 调用、站内资源匹配和行程保存。外部服务超时、返回非法 JSON 或资源部分匹配时，由服务层返回可解释的模板降级或候选结果。

HLD-FIG-13 COMP-UC12 AI 客服多轮对话组件级顺序图

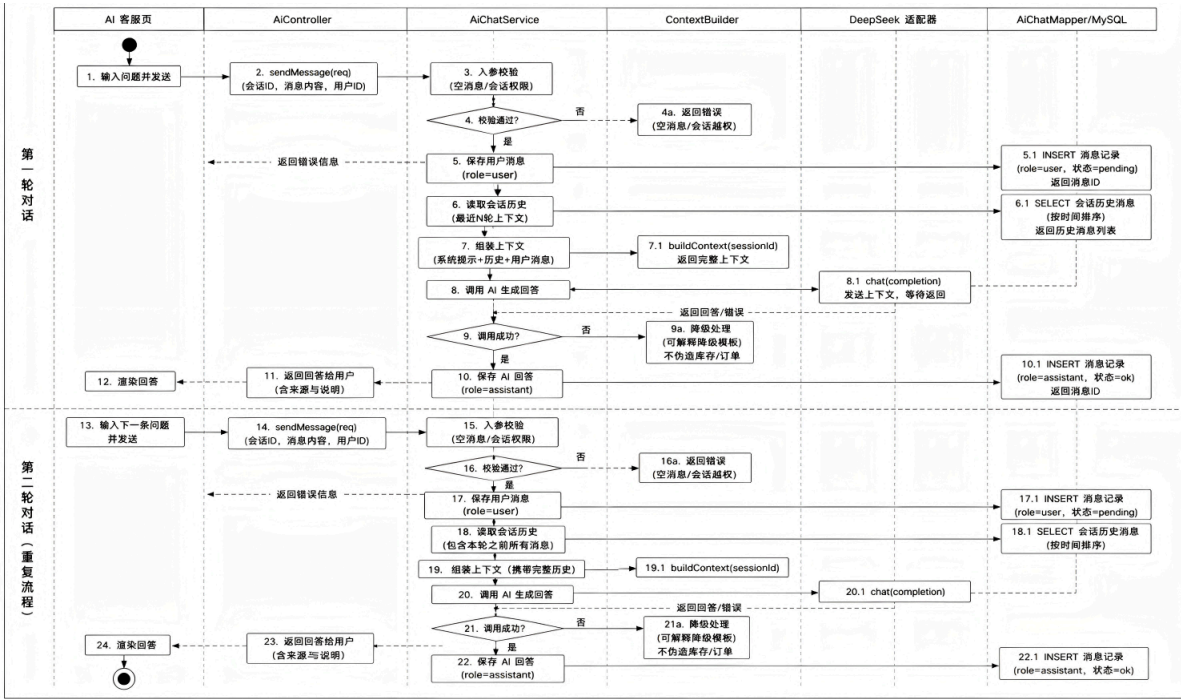


图 HLD-FIG-13 COMP-UC12 AI 客服多轮对话组件级顺序图

AiChatService 在每轮对话中校验会话归属、保存用户消息、读取历史记录并通过 ContextBuilder 组装上下文，再调用 DeepSeek 适配器生成回答。外部 AI 失败时返回降级提示，已保存的会话记录保持一致。

HLD-FIG-14 COMP-UC13 通知中心与站内私信组件级活动图

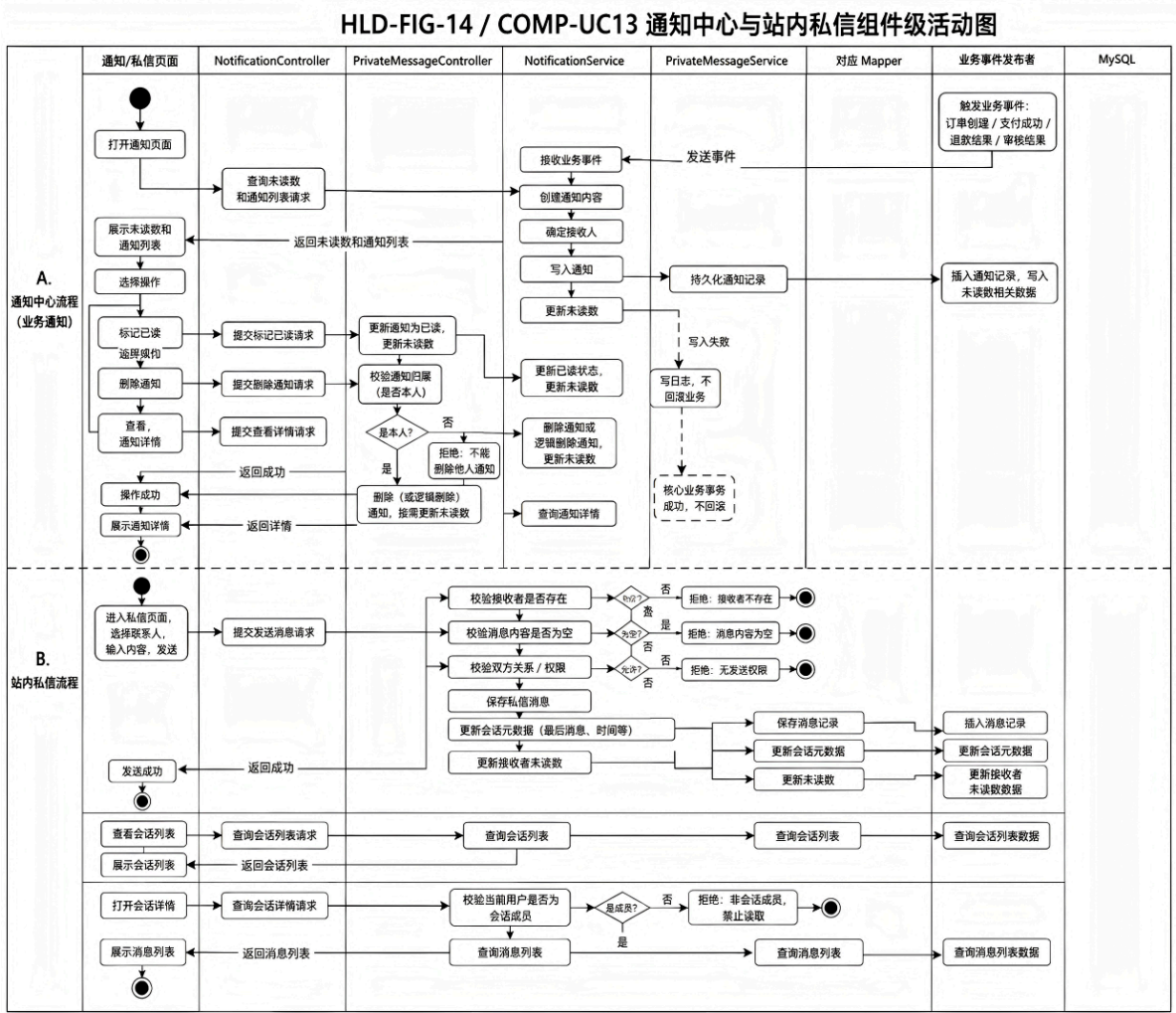


图 HLD-FIG-14 COMP-UC13 通知中心与站内私信组件级活动图

通知查询与维护接口由 AiController 暴露，经 AiServiceImpl 委托给 NotificationCenterServiceImpl 读写通知并维护未读数；业务事件发布者也通过 NotificationCenterService 写入通知。私信流程由 PrivateMessageController 和 PrivateMessageServiceImpl 完成发送、会话及消息查询。通知失败只记录日志而不回滚核心业务，私信读写必须校验会话成员权限。

HLD-FIG-15 COMP-UC14 游记发布、编辑、删除与审核组件级状态图

```

stateDiagram-v2
    [*] --> S0
    S0: S0 草稿编辑中 (未保存)
    S0 --> S1: 保存草稿
    S1: S1 草稿已保存 (draft) 作者可见
    S1 --> S2: 提交审核
    S2: S2 上传中 (uploading)
    S2 --> S3: 上传文件 (图片/视频等)
    S2 --> S2a: 上传失败
    S2a: S2a 上传失败 (upload_failed)
    S2a --> S1: 重新上传
    S3: S3 上传成功 (uploaded)
    S3 --> S4: 提交内容检测
    S4: S4 敏感词检测中 (checking)
    S4 --> S5: 检测通过 (无命中)
    S4 --> S4a: 命中敏感词
    S5: S5 待审核 (pending) 等待管理员审核
    S5 --> S8: 审核
    S4a: S4a 敏感词命中 (rejected_sensitivity) 驳回并提示修改
    S4a --> S12: 修改内容
    S4a --> S4: 修改后再次提交
    S12: S12 修改后重审 (pending) 版本+1
    S12 --> S4: 提交内容检测
    S6: S6 审核通过 (approved) 可发布
    S6 --> S8: 发布通知
    S8: S8 已发布 (published) 对外可见
    S8 --> S9: 状态落库 (status=published, audit_user, audit_time, version)
    S8 --> S10: S8 十天内
    S10: S10 通知作者 驳回原因
    S10 --> S11: S11 状态落库 (status=rejected, audit_user, audit_time, reason)
    S13: S13 已编辑待审 (editing_pending) 版本+1
    S13 --> S4: 提交审核
    S14: S14 删除审核中 (delete_pending)
    S14 --> S15: 审核
    S14 --> S16: 审核
    S15: S15 同意删除 (deleted)
    S15 --> S17: S17 状态落库 (status=deleted, deleted_by, time)
    S16: S16 拒绝删除 (delete_rejected)
    S16 --> S18: S18 通知作者 删除被拒绝
    S18 --> S19: S19 状态落库 (status=active, reason)
    S20: S20 违规下架中 (takedown_pending)
    S20 --> S21: 审核
    S21: S21 已下架 (takedowned)
    S21 --> S22: S22 通知作者 已下架
    S22 --> S23: S23 状态落库 (status=takedowned, reason, time)
    S24: S24 作者编辑
    S24 --> S13: 编辑并保存
    S25: S25 作者删除
    S25 --> S14: 删除请求
  
```

The diagram illustrates the state transitions for the Travelogue Release, Editing, Deletion, and Review Component. It is organized into columns representing different services: 游记编辑页 (Travelogue Editing Page), PostController, PostService, UploadService, SensitiveWordService, AdminController, NotificationService, and PostMapper/MySQL.

States and Transitions:

- S0 草稿编辑中 (未保存)** (Travelogue Editing Page): Initial state. Transition to **S1** on "保存草稿" (Save Draft).
- S1 草稿已保存 (draft) 作者可见** (PostController): Draft saved. Transition to **S2** on "提交审核" (Submit Review).
- S2 上传中 (uploading)** (PostService): Uploading. Transitions to **S3** on "上传文件 (图片/视频等)" (Upload File) and to **S2a** on "上传失败" (Upload Failed).
- S2a 上传失败 (upload_failed)** (PostService): Upload failed. Transition back to **S1** on "重新上传" (Re-upload).
- S3 上传成功 (uploaded)** (PostService): Upload successful. Transition to **S4** on "提交内容检测" (Submit Content Check).
- S4 敏感词检测中 (checking)** (SensitiveWordService): Checking for sensitive words. Transitions to **S5** on "检测通过 (无命中)" (Check Passed) and to **S4a** on "命中敏感词" (Sensitive Word Hit).
- S4a 敏感词命中 (rejected_sensitivity) 驳回并提示修改** (SensitiveWordService): Sensitive word hit. Transitions to **S12** on "修改内容" (Modify Content) and to **S4** on "修改后再次提交" (Resubmit after modification).
- S5 待审核 (pending) 等待管理员审核** (AdminController): Pending review. Transition to **S8** on "审核" (Review).
- S6 审核通过 (approved) 可发布** (AdminController): Review passed. Transition to **S8** on "发布通知" (Release Notification).
- S8 已发布 (published) 对外可见** (AdminController): Published. Transitions to **S9** (state database), **S10** (notification), and **S11** (reason database).
- S10 通知作者 驳回原因** (NotificationService): Notify author. Transition to **S11**.
- S11 状态落库 (status=rejected, audit_user, audit_time, reason)** (PostMapper/MySQL): Record rejection reason.
- S12 修改后重审 (pending) 版本+1** (PostService): Resubmit after modification. Transition to **S4** on "提交内容检测" (Submit Content Check).
- S13 已编辑待审 (editing_pending) 版本+1** (PostController): Edited and pending review. Transition to **S4** on "提交审核" (Submit Review).
- S14 删除审核中 (delete_pending)** (PostController): Deletion pending review. Transitions to **S15** on "审核" (Review) and to **S16** on "审核" (Review).
- S15 同意删除 (deleted)** (AdminController): Deletion approved. Transition to **S17**.
- S16 拒绝删除 (delete_rejected)** (AdminController): Deletion rejected. Transition to **S18**.
- S17 状态落库 (status=deleted, deleted_by, time)** (PostMapper/MySQL): Record deletion.
- S18 通知作者 删除被拒绝** (NotificationService): Notify author deletion rejected. Transition to **S19**.
- S19 状态落库 (status=active, reason)** (PostMapper/MySQL): Record deletion reason.
- S20 违规下架中 (takedown_pending)** (PostController): Violation takedown pending. Transition to **S21** on "审核" (Review).
- S21 已下架 (takedowned)** (AdminController): Takedowned. Transition to **S22** on "通知作者 已下架" (Notify author takedowned).
- S22 通知作者 已下架** (NotificationService): Notify author takedowned. Transition to **S23**.
- S23 状态落库 (status=takedowned, reason, time)** (PostMapper/MySQL): Record takedown.
- S24 作者编辑** (PostController): Author editing. Transition to **S13** on "编辑并保存" (Edit and Save).
- S25 作者删除** (PostController): Author deletion. Transition to **S14** on "删除请求" (Deletion Request).

PostService 协调文件上传、敏感词检测、后台审核和通知组件，管理游记从草稿到待审核、已发布、已驳回、已下架或已删除的状态变化。状态、审核人、原因和版本信息统一持久化，重复审核保持幂等。

HLD-FIG-16 COMP-UC15 社区点赞、收藏与评论组件级活动图

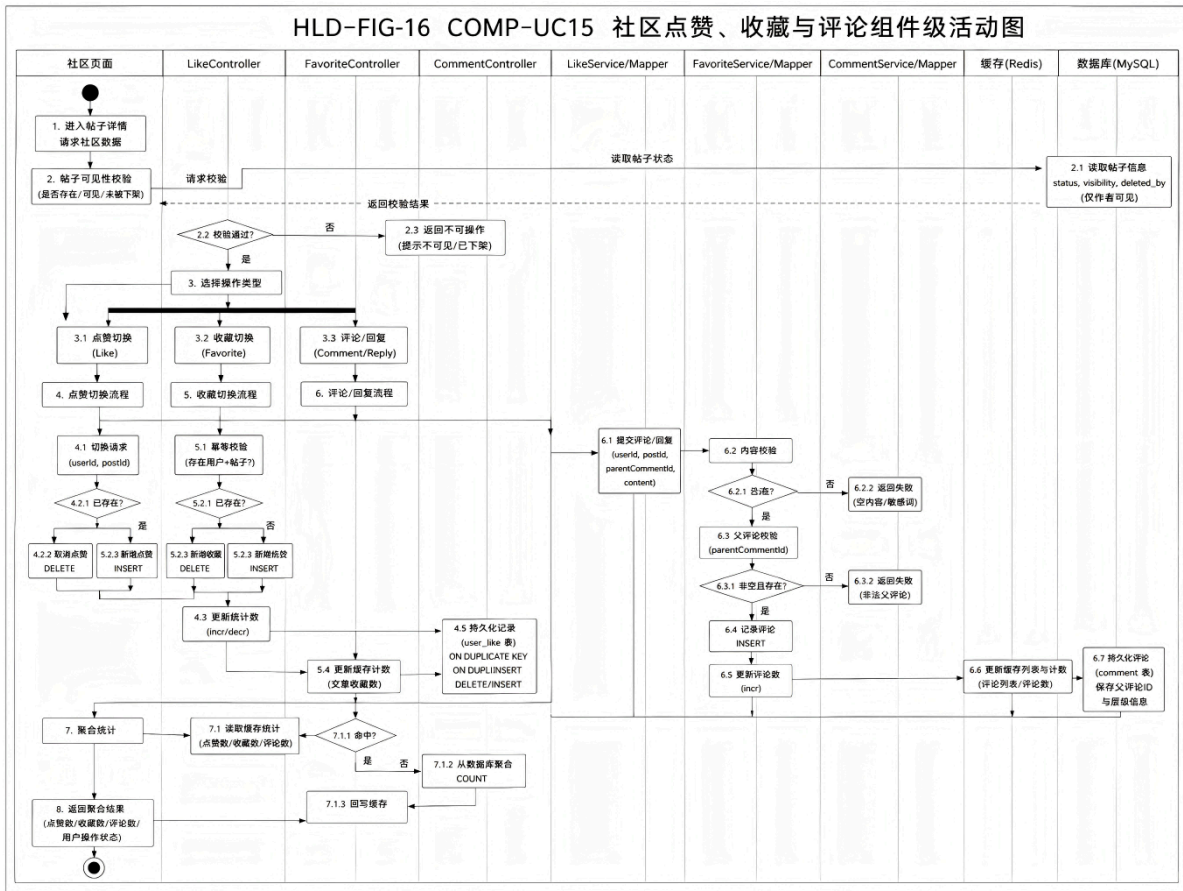


图 HLD-FIG-16 COMP-UC15 社区点赞、收藏与评论组件级活动图

社区页面通过点赞、收藏和评论 Controller 调用各自 Service 与 Mapper，完成关系切换、树状评论保存及统计返回。唯一约束保证点赞和收藏幂等，缓存更新失败时以数据库结果校准计数。

HLD-FIG-17 COMP-UC16 常用旅客管理与使用组件级活动图

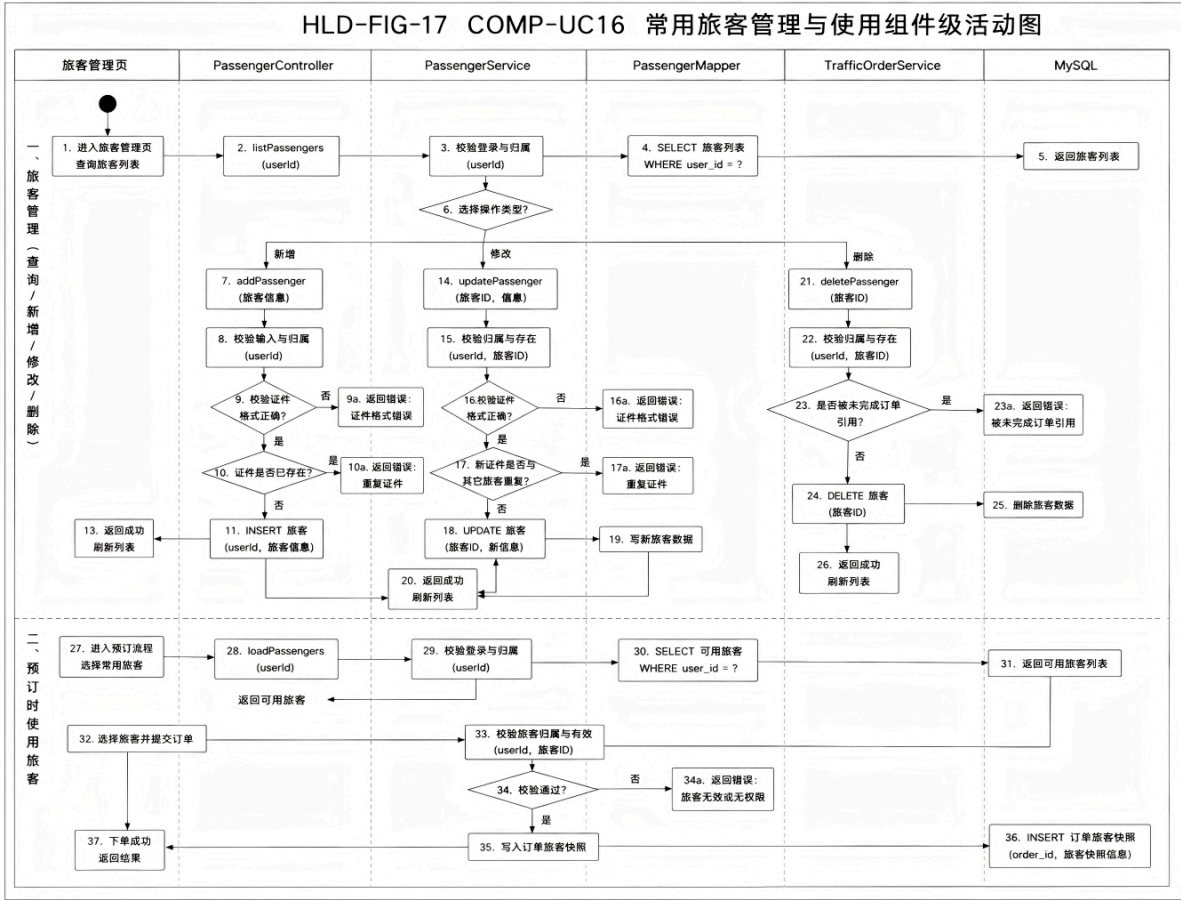


图 HLD-FIG-17 COMP-UC16 常用旅客管理与使用组件级活动图

PassengerController 和 PassengerService 完成旅客查询、新增、修改及删除，PassengerMapper 负责持久化。预订时 TrafficOrderService 校验旅客归属和有效性，并把旅客快照写入订单，防止后续资料变化影响历史订单。

HLD-FIG-18 COMP-UC17 用户主页与关注关系组件级活动图

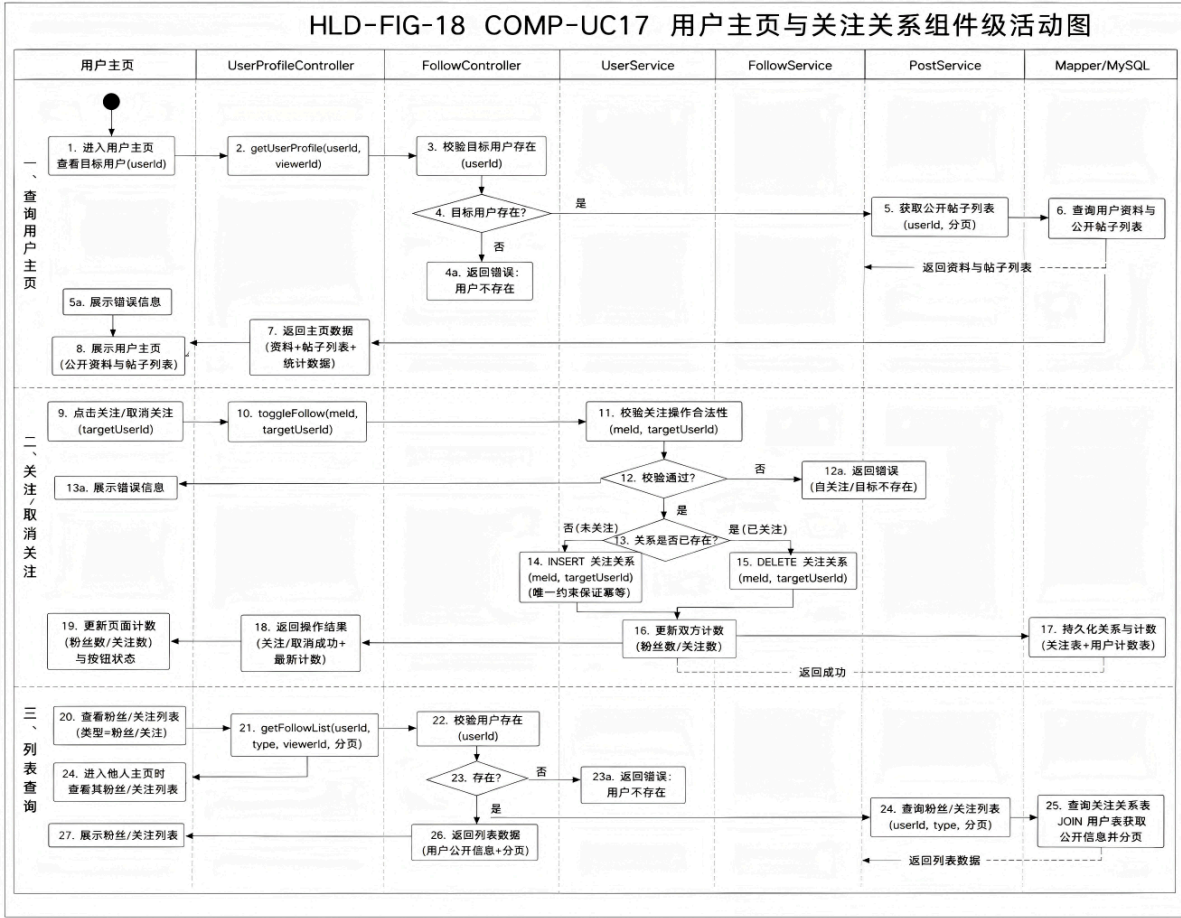


图 HLD-FIG-18 COMP-UC17 用户主页与关注关系组件级活动图

UserProfileController 聚合公开资料和帖子，FollowController 与 FollowService 负责关注关系切换和列表查询。唯一约束保证关注操作幂等，计数更新与关系持久化保持一致，同时对外裁剪隐私字段。

HLD-FIG-19 COMP-UC18 管理后台资源、订单与用户管理组件级活动图

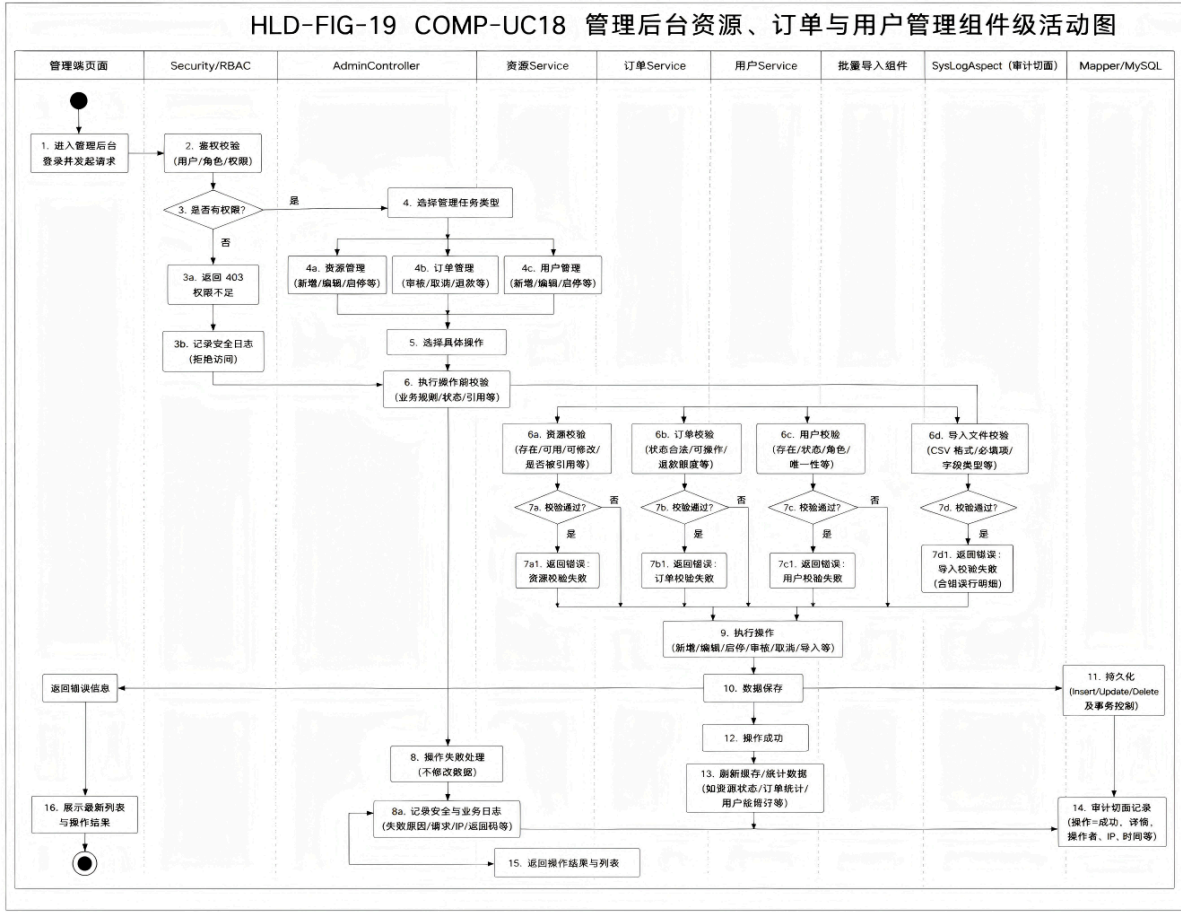


图 HLD-FIG-19 COMP-UC18 管理后台资源、订单与用户管理组件级活动图

管理端请求首先通过 Security/RBAC，再由 AdminController 分派到资源、订单、用户或批量导入组件。业务校验、数据持久化和 SysLogAspect 审计共同构成管理闭环，失败操作同样保留必要的安全日志。

HLD-FIG-20 COMP-UC19 内容安全、举报处理与可观测性组件级活动图

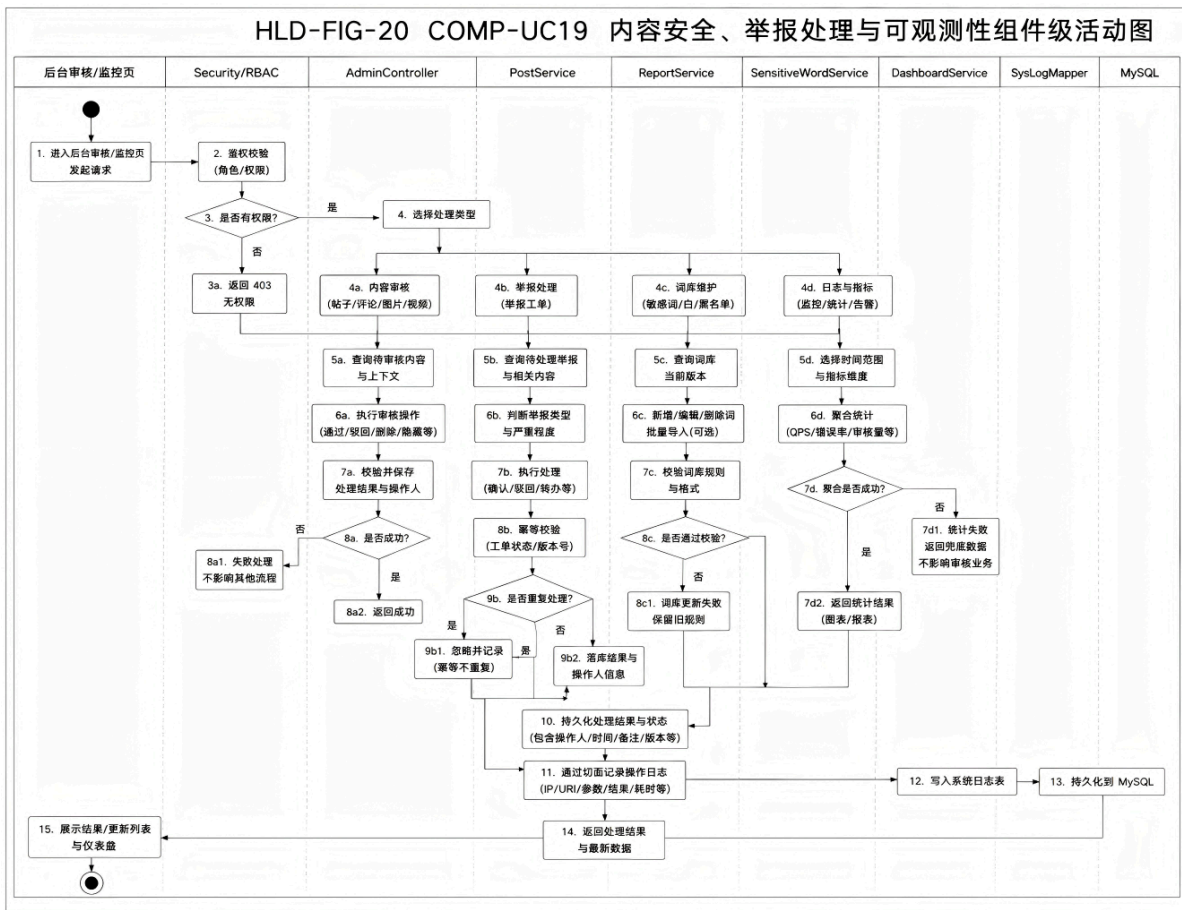


图 HLD-FIG-20 COMP-UC19 内容安全、举报处理与可观测性组件级活动图

后台审核与监控页面通过 AdminController 调用内容、举报、敏感词和指标服务，处理结果与操作人统一写入数据库和系统日志。重复工单按终态幂等处理，词库或统计失败不影响其他审核业务继续运行。